

urdu-slm

Deep Dive

A small Urdu language model trained from scratch:
the whole stack, from raw public text to a decoder-only transformer

This document explains every part of the **urdu-slm** repository to a reader who has never trained a language model. It defines each technical term the first time it appears, prefers a diagram to a paragraph wherever a diagram can carry the idea, and separates what was verified by running the code from what was read in the README. Where the README and the code disagree, the code wins, and the disagreement is recorded rather than smoothed over. Section 11 collects every such finding in one place.

Contents

1	Orientation: what this repository actually is	3
1.1	A necessary note about the script	3
2	Part I: Foundations, assuming nothing	3
2.1	What a language model is	4
2.2	Tokens: why models do not read letters	4
2.3	Why tokenizing Urdu is not like tokenizing English	5
2.4	What a transformer is	6
3	Part II: The system at a glance	8
3.1	The repository, file by file	9
4	Part III: The data pipeline	11
4.1	<code>sources.py</code> : one reader per format	11
4.2	<code>normalize.py</code> : making Urdu canonical	12
4.3	<code>dedup.py</code> : the most interesting code in the repository	14
4.4	<code>common.py</code> : the plumbing that makes stages resumable	16
4.5	<code>build_corpus.py</code> : the orchestrator	17
4.6	What the pipeline actually produced	17
5	Part IV: The tokenizer	19
5.1	<code>train_tokenizer.py</code> : 56 lines, several real decisions	19
5.2	<code>tokenize_corpus.py</code> : text to a flat array of numbers	20
5.3	The 5.23x claim, verified	21
6	Part V: The model	24
6.1	<code>config.py</code> : one dataclass, four presets	24
6.2	The block, drawn	26
6.3	RMSNorm	27
6.4	Rotary position embeddings (RoPE)	27
6.5	Attention	29
6.6	SwiGLU	30
6.7	UrduLM: assembly, tying, and initialization	31
6.8	The tests	33
7	Part VI: Training	34
7.1	<code>TokenLoader</code> : batches from a memory-mapped stream	34
7.2	The optimizer and the schedule	35
7.3	Token-count budgeting	36
7.4	Gradient accumulation, and mixed precision	36
7.5	Checkpointing and resume	38
7.6	The configs	39
7.7	Distributed training	40
8	Part VII: Evaluation	41
8.1	<code>eval/perplexity.py</code> : the primary metric	41
8.2	<code>eval/script_mix.py</code> : is it even writing Urdu?	42
8.3	<code>eval/cloze.py</code> : does it know anything?	42
8.4	<code>eval/plot_loss.py</code> and <code>sample.py</code>	44

9	Part VIII: What actually happened when it ran	46
9.1	The tiny CPU proof run	46
9.2	Base v1: the first real GPU run	46
9.3	Base v2: the headline model	47
9.4	v1 against v2, and the honest reading	48
10	Part IX: The planned medium run	50
11	Part X: Honest findings, collected	51
11.1	Substantive: worth fixing before the next run	51
11.2	Real but contained	51
11.3	Documentation and hygiene	52
11.4	What is genuinely strong	52
12	Part XI: Reproduction, and what was verified	54
12.1	Dependency roles	54
12.2	What was verified for this document, and how	55
12.3	Running it yourself	55

1 Orientation: what this repository actually is

`urdu-slm` builds a language model for Urdu starting from nothing but publicly downloadable text. It is not a fine-tune of somebody else's model, and it does not import modeling code from the `transformers` library. Every layer of the network is written out in PyTorch in one 186-line file, and every stage that turns a raw Wikipedia dump into training-ready numbers is written out in the `pipeline/` package.

The whole project in five moves

1. **Collect** raw Urdu text from public sources that need no login.
2. **Clean** it: normalize the spelling, throw away non-Urdu and boilerplate, and remove duplicates.
3. **Build a tokenizer**, a scheme for cutting Urdu text into reusable chunks, trained on that cleaned Urdu rather than borrowed from an English model.
4. **Train a transformer** to predict the next chunk, over and over, on a rented GPU.
5. **Measure** it honestly, and report the numbers that came out, including the unflattering ones.

The repository ships three trained artifacts: a 13M-parameter model trained on a CPU as a proof that the pipeline works end to end, and two 124M-parameter models trained on a rented A100 GPU (`base` on 2026-07-07 and `base_v2` on 2026-07-08). The per-step training logs for all three are committed, which is what made much of this document checkable rather than merely repeated.

1.1 A necessary note about the script

Why you will not see a single Urdu character in this PDF

Urdu is written in the Perso-Arabic script: right to left, with letters that change shape depending on their neighbours. This document is typeset with `pdflatex`, which on this machine has no font capable of rendering that script and no engine for right-to-left shaping. Pasting Urdu text into a code listing here is not a cosmetic problem, it is a hard build failure: the `listings` package reads raw bytes and the input encoder aborts on the first multi-byte character.

So every Urdu string in this document is handled one of three ways: transliterated into Latin letters, described in English, or given as Unicode codepoints. The real script lives in the repository's own README and in `evaldata/cloze.jsonl`, which render correctly in any modern editor. Nothing has been hidden; it simply cannot be drawn here, and an honest sentence beats a document that will not build.

2 Part I: Foundations, assuming nothing

If you already know what a decoder-only transformer is, skip to Section 3. Nothing in this part is specific to this repository; it is the vocabulary the rest of the document uses.

2.1 What a language model is

Jargon: Language model

A program that, given a stretch of text, assigns a probability to every possible next piece of text. That is the entire definition. It does not "understand" in any agreed sense; it estimates how likely each continuation is, given everything it has read during training.

Suppose the text so far is "the capital of Pakistan is". A language model outputs a number for every entry in its vocabulary: perhaps 0.6 for "Islamabad", 0.05 for "Karachi", and a vanishingly small number for "bicycle". Those numbers must sum to one across the whole vocabulary. To generate text you sample from that distribution, append what you drew, and repeat.

Jargon: Training

The process of adjusting the model's internal numbers (its *parameters*) so that the probabilities it assigns to real text are as high as possible. You show it real sentences with the answer hidden, measure how surprised it was by the true next piece, and nudge every parameter slightly in the direction that would have reduced that surprise.

Jargon: Parameter

One adjustable number inside the model. The **base** model here has 123.7 million of them. They start as random noise and end, after training, holding everything the model knows. "124M parameters" is a statement about the model's capacity to store patterns, not about its size on disk (though the two are related: 123.7M parameters at 4 bytes each is the 495MB of `runs/base_v2/model.pt`).

Jargon: Loss, and cross-entropy

The number that measures "how surprised the model was". This project uses *cross-entropy loss*, measured in *nats*. If the model gave the true next token probability p , the loss for that token is $-\ln p$. A confident correct prediction (p near 1) gives a loss near 0; assigning the truth a probability of one-in-a-thousand gives a loss of about 6.9. Training is the search for parameters that make the average loss small.

Jargon: Perplexity

Loss made human-readable: $\text{perplexity} = e^{\text{loss}}$. It answers "on average, how many equally-likely options did the model feel it was choosing between?" A perplexity of 28.93 (this project's best model) means the model was about as uncertain as someone picking uniformly among 29 choices. Lower is better. A perplexity equal to the vocabulary size (32,000) is a model that has learned nothing.

2.2 Tokens: why models do not read letters

A model cannot consume text directly; it consumes integers. The mapping between the two is where this project does its most distinctive work.

Jargon: Token

The unit a language model reads and writes. Not a character, and usually not a word: a frequently occurring *chunk* of characters. In an English model, "the" is typically one token, and an unusual word like "tokenization" might be three ("token", "iz", "ation"). The model's

vocabulary is the fixed list of every chunk it knows, here 32,000 of them.

Jargon: Tokenizer

The program that cuts text into tokens and back again. It is trained, separately from and before the model, by looking at a corpus and deciding which chunks are worth having their own vocabulary entry. Once trained it is frozen: the model is built around one specific tokenizer and cannot be used with another.

Why does chunking matter so much? Because every cost in the system is counted in tokens. The GPU bill is per token. The model's context window, how much text it can see at once, is measured in tokens (1024 here). If a tokenizer needs five times as many tokens to express the same sentence, then the same GPU budget reads one fifth as much text, and the model's 1024-token window holds one fifth as much meaning.

Byte-Pair Encoding (BPE), the algorithm used here

Start with a vocabulary of single characters (or, here, single bytes). Then repeat: scan the corpus, find the pair of adjacent symbols that occurs most often, and merge it into one new symbol added to the vocabulary. Do this until the vocabulary reaches the target size (32,000). Common sequences in the training text end up as single tokens; rare ones stay split. The tokenizer is therefore a direct fingerprint of the language it was trained on.

2.3 Why tokenizing Urdu is not like tokenizing English

This is the heart of the project's motivation, so it is worth being precise.

Text is stored as Unicode codepoints, encoded into bytes with UTF-8. UTF-8 is deliberately biased toward English: every ASCII character (a to z, digits, punctuation) costs exactly **one byte**. Urdu's letters live in the Arabic block, codepoints U+0600 to U+06FF, and every one of them costs **two bytes**.

Now consider a byte-level BPE tokenizer trained on English web text, which is what GPT-2's is. Its 50,257 merges were chosen to compress English. It has a token for "the" and a token for "Pakistan". It has essentially no merges covering Urdu byte sequences, because it barely saw any. So when it meets Urdu it falls back to its base alphabet and emits roughly one token per byte, and each Urdu letter is two bytes.

The compounding penalty

An English tokenizer on Urdu pays twice over. First, Urdu characters cost two UTF-8 bytes each instead of one. Second, no merges fire, so those bytes never combine back into larger chunks. The result is a tokenizer that spends more than two tokens per Urdu *character*, while a tokenizer trained on Urdu spends about one token per four characters. The measured gap is Section 5.3, and it is 5.23x.

Two further properties of Urdu shape the pipeline in Section 4:

- **Encoding variants.** The same visible Urdu letter can be written with different codepoints, because Urdu borrows the Arabic script and both the Arabic and Persian-adapted forms circulate. The letter "kaf" appears as U+0643 (Arabic) or U+06A9 (Urdu keheh); "yeh" as U+064A (Arabic) or U+06CC (Farsi). To a human these look identical. To a BPE trainer they are unrelated symbols, so it would waste vocabulary learning both spellings of every word. `normalize.py` folds them; see Section 4.2.

- **Rich morphology.** Urdu inflects heavily, so a word-level vocabulary would explode. Subword BPE handles this naturally by learning stems and endings.

2.4 What a transformer is

The model itself is a *decoder-only transformer*. Taking that apart:

Jargon: Neural network

A stack of layers, each of which multiplies its input by a matrix of parameters and applies a simple non-linear function. Numbers go in at the bottom, get reshaped layer by layer, and predictions come out at the top. Training adjusts the matrices.

Jargon: Embedding

The lookup table that turns a token id (an integer) into a vector of numbers the network can work with. Here it maps each of 32,000 token ids to a vector of 768 numbers for the **base** model. Those $32,000 \times 768 = 24.6$ million numbers are themselves parameters, learned during training. This table's size is a recurring theme in Section 6.1.

Jargon: Attention

The mechanism that lets a position in the sequence look at other positions. For each position the model computes three vectors: a *query* (what am I looking for?), a *key* (what do I offer?), and a *value* (what do I pass along if chosen?). Each position compares its query against every key, turns the comparison scores into weights that sum to one, and takes that weighted average of the values. This is how the word "it" finds the noun it refers to twelve words back.

Jargon: Causal (masked) attention, and "decoder-only"

"Causal" means a position may attend only to itself and to positions *before* it, never after. This is what makes next-token prediction a fair game: when predicting position 50, the model must not be allowed to peek at position 51, which is the answer. "Decoder-only" describes an architecture built entirely from causal blocks, with no separate encoder. Every modern text-generating model (the GPT family, Llama, and this one) is decoder-only.

Figure 1 shows what the mask actually does. This property is not merely asserted in the repository; `tests/test_model.py` pins it with a direct experiment, described in Section 6.8.

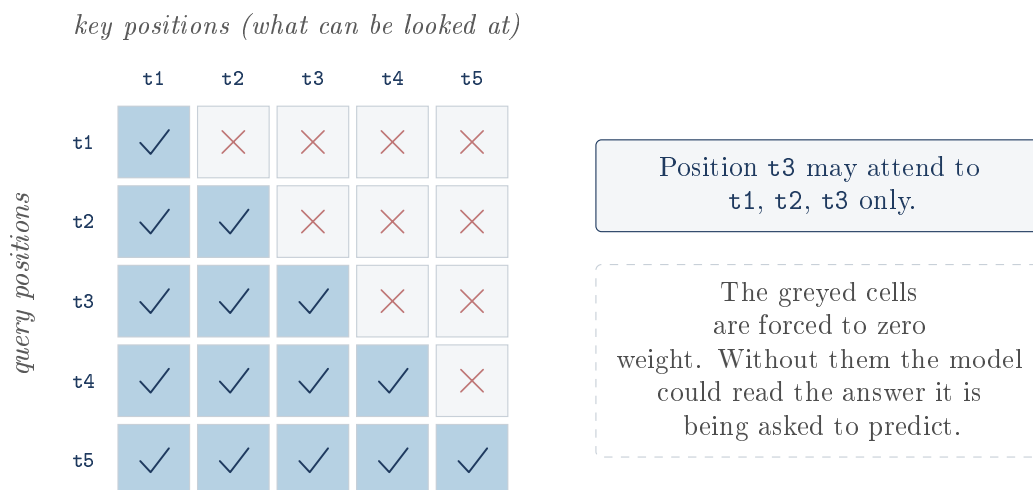


Figure 1: Causal masking. Each row is a position asking a question; each column is a position that could answer. Everything above the diagonal is forbidden.

Jargon: Context window (block size)

The maximum number of tokens the model can attend over at once, `max_seq_len` in the code: 512 for `tiny`, 1024 for the others. Attention cost grows with the square of this number, which is why it is a deliberate, budgeted choice rather than "as large as possible".

3 Part II: The system at a glance

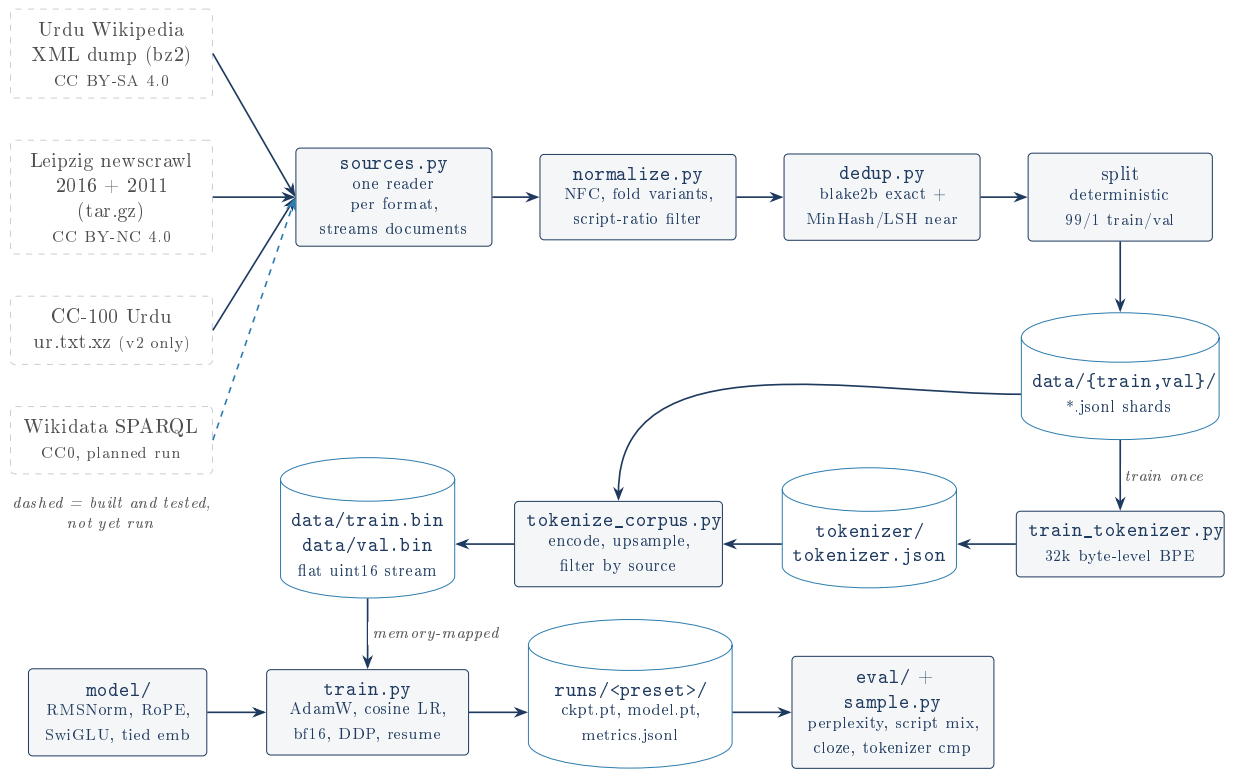


Figure 2: End-to-end architecture. Solid arrows are paths that have been executed and produced the committed artifacts. The Wikidata source is wired and unit-tested but has not been used in any training run.

3.1 The repository, file by file

```

urdu-slm
├── pipeline/ raw text → token stream
│   ├── sources.py readers: wiki dump, Leipzig tar, plain text, Wikidata
│   ├── normalize.py NFC, variant folding, script-ratio language ID
│   ├── dedup.py exact blake2b + vectorized MinHash LSH
│   ├── common.py ShardWriter, Stage resume markers, iter_jsonl
│   ├── build_corpus.py orchestrates extract → dedup → split
│   ├── train_tokenizer.py 32k byte-level BPE
│   ├── tokenize_corpus.py encode to uint16 .bin, upsample, filter
│   └── fetch_wikidata.py SPARQL fact triples (standalone)
├── model/
│   ├── config.py ModelConfig dataclass + 4 size presets
│   └── transformer.py the entire network, 186 lines
├── train.py the pretraining loop
├── sample.py generate continuations from a checkpoint
├── eval/
│   ├── _shared.py load_model helper
│   ├── perplexity.py held-out loss on a .bin split
│   ├── script_mix.py share of generated chars in Arabic script
│   ├── cloze.py few-shot fill-in-the-blank harness
│   ├── tokenizer_compare.py ours vs GPT-2 compression
│   └── plot_loss.py metrics.jsonl → PNG curve
├── configs/ tiny.yaml small.yaml base.yaml medium.yaml
├── evaldata/cloze.jsonl 95 hand-written Urdu items
├── tests/ test_model.py (5), test_pipeline.py (8)
├── runs/ tiny/, base/, base_v2/ + metrics.jsonl (LFS weights)
├── data/ val.bin, corpus_stats.json, sample/ (train.bin gitignored)
└── docs/ GPU_RUN.md, DATA_LICENSES.md, explainers/

```

Figure 3: Directory structure with each file’s job. The whole project is about 2,800 lines of Python including tests and docs.

Each stage writes its output to disk and never holds the corpus in memory. Sources *stream* (Python generators, one document at a time), writers *shard* (fixed size JSONL files), stages *mark themselves done* (a marker file), and training *memory-maps* the token stream. A 1.7GB corpus is processed on a machine that never allocates 1.7GB for it. This is the single most consistent design decision in the repository.

4 Part III: The data pipeline

The `pipeline/` package answers one question: how do you turn a 455MB compressed Wikipedia dump and a pile of news sentences into a clean, deduplicated, tokenized stream of numbers, on a laptop, without ever loading it all into memory?

4.1 `sources.py`: one reader per format

Every source is a Python *generator* yielding one dictionary per document, shaped `{text, source}`. This uniformity is what lets the rest of the pipeline stay source-agnostic.

Jargon: Generator (Python)

A function that produces values one at a time with `yield` instead of building a whole list. The caller consumes them in a loop, and only one value exists in memory at a time. This is the mechanism behind the streaming claim: `read_wiki_dump` on a 455MB dump never holds more than one article.

4.1.1 `read_wiki_dump`: parsing a MediaWiki dump

A Wikipedia dump is a single enormous XML file, bz2-compressed. The reader uses `ET.iterparse`, which fires an event as each element *closes*, rather than building a tree of the whole document.

```

1 for event, elem in ET.iterparse(fh, events=("end",)):
2     tag = elem.tag.split("}")[-1]
3     if tag == "title": title = elem.text
4     elif tag == "ns": ns = elem.text
5     elif tag == "text": text = elem.text
6     elif tag == "page":
7         if ns == "0" and text and title and not _REDIRECT_RE.match(text):
8             body = _strip_wikitext(text)
9             ...
10    title, text, ns = None, None, None
11    elem.clear()           # <- releases the parsed element's memory
```

Listing 1: The streaming discipline in `read_wiki_dump`

Three details carry real weight:

- `elem.clear()` is what makes this constant-memory. Without it, `iterparse` still accumulates every parsed element and the process grows to the size of the dump.
- `tag.split("}")[-1]` strips the XML namespace. MediaWiki tags do not arrive as a bare `title`; they arrive with the schema URL wrapped in braces in front of the tag name, as shown below. Splitting on the closing brace and taking the last piece discards the namespace and leaves the plain tag name.

```

1 {http://www.mediawiki.org/xml/export-0.10/}title -> title
```

Listing 2: The namespaced tag that the split has to strip

- `ns == "0"` keeps only article-namespaced pages, dropping Talk pages, User pages, Templates and Categories. `_REDIRECT_RE` then drops redirect stubs, matching `#REDIRECT` in English *and* two Urdu spellings of the same directive.

`_strip_wikitext` hands the wikitext to `mwparserfromhell`, an external library that understands MediaWiki markup, and calls `strip_code`. It then drops any surviving line starting with a pipe, exclamation mark, or brace, which are table and infobox leftovers the parser does not fully clean.

Jargon: Wikitext

Wikipedia's own markup language: `[[links]]`, `{{templates}}`, table syntax. It is famously irregular, which is why parsing it is delegated to a dedicated library rather than attempted with regular expressions.

4.1.2 The other three readers

- `read_leipzig_tar` opens a `.tar.gz`, scans members for the one ending `-sentences.txt`, and splits each line on the first tab, since Leipzig's format is `<id>\t<sentence>`. If no such member exists it returns silently, yielding nothing.
- `read_plain_text` handles the CC-100 layout: consecutive non-empty lines form one document, a blank line ends it. It transparently opens `.xz`, `.gz` or plain files, and passes `errors="ignore"` so a single malformed byte in a 9-million-document crawl cannot kill the run.
- `read_wikidata_facts` reads the JSONL written by `pipeline/fetch_wikidata.py` and renders each subject/relation/object triple into a declarative Urdu sentence using two fixed templates per relation.

Why the Wikidata templates alternate by index, not at random

`template = templates[i % len(templates)]` picks the phrasing from the row's line number. Using `random.choice` would give the same lexical variety but a different corpus on every run, which would silently break reproducibility. Deterministic alternation buys the variety without the cost. This is a small decision, well made.

4.2 normalize.py: making Urdu canonical

This module is where the project's Urdu-specific knowledge is concentrated, and it is the part a generic "train a GPT" tutorial does not have at all.

4.2.1 The folding map

Section 2.3 explained why encoding variants must be collapsed. The code does it with a compiled alternation regex over a nine-entry map. Because the actual characters cannot be printed here, Table 1 gives the map by codepoint and name.

From	Name	To	Why
U+0643	Arabic Kaf	U+06A9 (Urdu Keheh)	same letter, Arabic spelling
U+064A	Arabic Yeh	U+06CC (Farsi Yeh)	same letter, Arabic spelling
U+0649	Alef Maksura	U+06CC (Farsi Yeh)	same letter, Arabic spelling
U+06C0	Heh with Hamza Above	U+06C1 (Heh Goal)	common variant
U+200B	Zero-width space	(removed)	invisible, no meaning
U+200D	Zero-width joiner	(removed)	invisible, no meaning
U+FEFF	BOM	(removed)	invisible, no meaning
U+00A0	Non-breaking space	U+0020 (space)	normalize whitespace
U+0640	Tatweel / kashida	(removed)	decorative elongation only

Table 1: The variant-folding map from `_FOLD`, given by codepoint because the glyphs cannot be typeset here.

The zero-width non-joiner is deliberately kept

U+200C, the zero-width non-joiner, is *not* in the removal list, while its neighbours U+200B and U+200D are. This is not an oversight. In Urdu the ZWNJ carries real orthographic meaning: it stops two letters from joining where joining would change the word. Deleting it would corrupt text. `tests/test_pipeline.py` pins this exact behaviour with an assertion that a ZWNJ survives normalization while a tatweel next to it does not. That is a well-chosen test.

The module’s docstring also records a decision worth defending out loud: *harakat* (short-vowel diacritics) are **not** stripped. They are rare in running Urdu but meaningful when present, for instance in religious quotation and dictionaries, so removing them would corrupt that minority of text. Many pipelines strip them by reflex.

4.2.2 NFC normalization

Jargon: Unicode normalization, NFC

The same visible character can sometimes be encoded either as one codepoint or as a base character plus a combining mark. NFC (Normalization Form Composed) rewrites text so the composed form always wins. Without it, two identical-looking strings can compare as different, which would defeat exact deduplication and split the tokenizer’s vocabulary. `normalize()` applies NFC *first*, then folds variants.

4.2.3 Language identification without a model

`arabic_ratio(text)` counts what share of the *letters* (characters where `c.isalpha()` is true, so digits and punctuation are excluded) fall in one of five Unicode ranges: Arabic (U+0600 to U+06FF), Arabic Supplement, Arabic Extended-A, and the two Presentation Forms blocks.

The reasoning behind skipping fastText

The obvious tool is fastText’s `lid.176` language-ID model. It is 126MB and adds a download plus a dependency. The module argues that Urdu shares its script only with other Perso-Arabic languages, none of which appear at volume in these sources (a Wikipedia dump already filtered to `ur`, and Leipzig corpora already tagged `urd`), so the real contaminant is embedded English, which a script ratio catches directly. For *these* sources that reasoning holds. The README is candid that a mixed-script web crawl would need real token-level language ID, and CC-100 (added in v2) is exactly such a crawl.

`keep_line()` then applies four filters in order, all cheap tests first:

Filter	Threshold	Catches
<code>len(text) < MIN_CHARS</code>	40	fragments, headings
<code>len(text.split()) < MIN_WORDS</code>	6	captions, list items
<code>arabic_ratio(text) < MIN_ARABIC_RATIO</code>	0.65	English and other-script lines
<code>alpha share of length < 0.5</code>	0.5	tables, infobox leftovers, digit soup

Table 2: The boilerplate and language filter, `normalize.keep_line`.

The README records that writing the test for this surfaced a bug in the author’s own expectation, not in the code: a mixed Urdu/English phrase is 4 Arabic letters of 16, so 0.25 rather than the 0.5 first assumed. The test now pins the real value with a range assertion. That is the test doing its job.

4.3 dedup.py: the most interesting code in the repository

Deduplication matters here more than in a typical corpus, for a reason specific to this language: Urdu Wikipedia is largely bot-generated stub articles. The pipeline extracted 2.46 million paragraphs from it, far more than the article count would suggest, and a large share are near-identical templated text. Training on them would teach the model to recite templates.

4.3.1 Exact duplicates

`exact_key` hashes the normalized text with `blake2b` truncated to 8 bytes, and a Python `set` holds the keys. Storing a 64-bit digest instead of the text is what keeps 3.7 million documents' worth of keys in memory.

Jargon: Hash function

A function that maps data of any size to a fixed-size number, such that the same input always gives the same output and different inputs almost never collide. Comparing hashes is a cheap stand-in for comparing the full text.

4.3.2 Near duplicates: MinHash and LSH

Exact hashing misses the real problem: two paragraphs differing by one character have completely unrelated hashes. That is what MinHash is for.

Jargon: Shingle

A short overlapping substring. `_shingles(text, k=5)` strips all whitespace and takes every 5-character window, deduplicated into a set. "abcdefg" gives {abcde, bcdef, cdefg}. Two documents that share most of their shingles are near-duplicates.

Jargon: Jaccard similarity

The size of the intersection of two sets divided by the size of their union. 1.0 means identical sets, 0.0 means no overlap. Here the sets are the two documents' shingles, and the pipeline's threshold is 0.8.

Jargon: MinHash

A trick for estimating Jaccard similarity without comparing the sets. Apply a random permutation to the universe of possible shingles and record only the *minimum* element of each set under it. The probability that two sets agree on that minimum is exactly their Jaccard similarity. Repeat with 64 different permutations and you get a 64-number *signature*; the share of positions where two signatures agree estimates their Jaccard. Constant size, regardless of document length.

Jargon: LSH (Locality-Sensitive Hashing)

Even with signatures, comparing every new document against every kept one is quadratic. LSH splits each 64-number signature into bands, hashes each band into a bucket, and only compares documents that collide in at least one band. Similar documents collide with high probability; dissimilar ones almost never do. This turns "compare against everything" into "look in a few buckets".

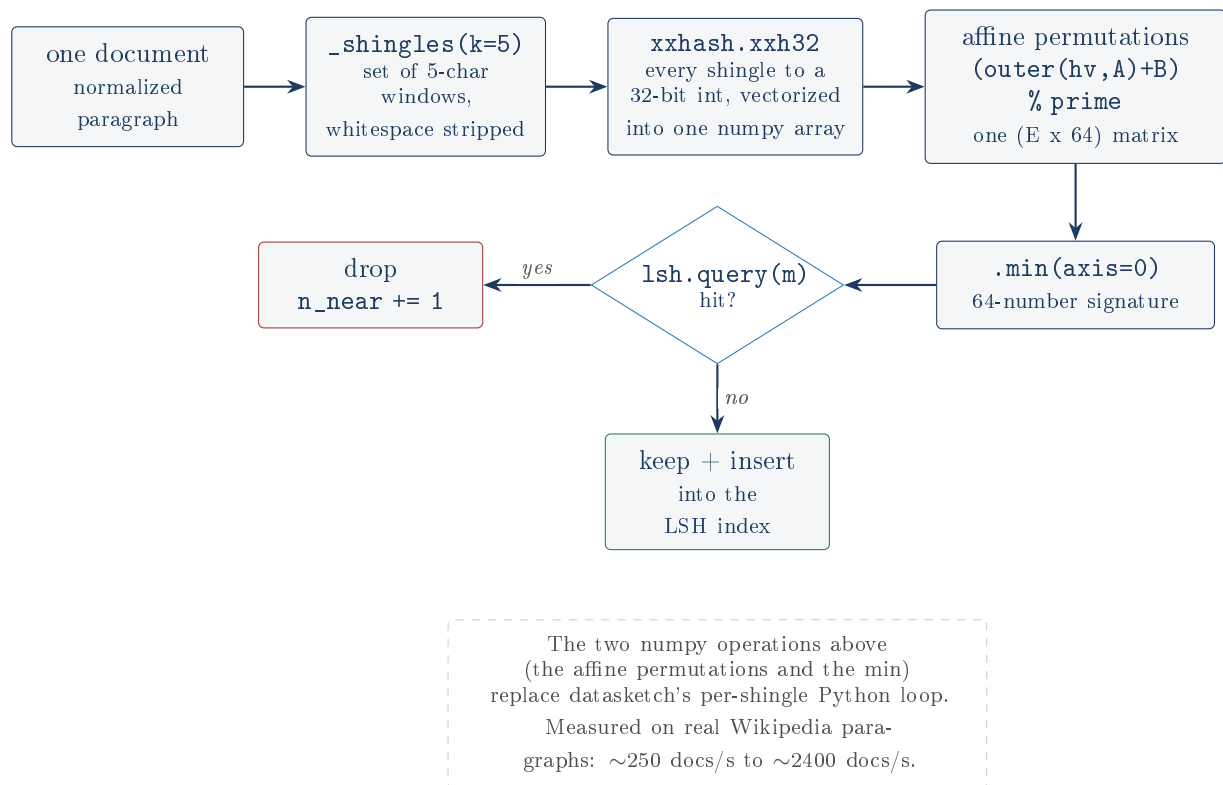


Figure 4: The near-duplicate path for a single document. Exact blake2b dedup runs first and short-circuits before any of this.

4.3.3 The vectorization, and why it was necessary

This is the repository's best engineering story, and it is worth being able to tell it precisely.

```

1  _seed = MinHash(num_perm=64, seed=1, scheme="affine32")
2  _A, _B = _seed.permutations          # affine coefficients, shape (64,)
3
4  def build_minhash(text, num_perm=_NUM_PERM):
5      m = MinHash(num_perm=num_perm, seed=1, permutations=(_A, _B), scheme="affine32")
6      shs = _shingles(text)
7      if not shs:
8          return m
9      hv = np.fromiter((xxhash.xxh32(s).intdigest() for s in shs),
10                      dtype=np.uint64, count=len(shs))
11      phv = (np.outer(hv, _A) + _B) % _MERSENNE      # (E, 64) in one op
12      phv = np.bitwise_and(phv, _MAX_HASH)
13      m.hashvalues = phv.min(axis=0).astype(np.uint64) # (64,)
14      return m

```

Listing 3: The hot path, rewritten

The library's own API is `MinHash.update(shingle)`, called once per shingle. That loops in Python and runs a tiny numpy operation per call, so per-call overhead dominates and the work never reaches numpy's fast path. Benchmarked on real Wikipedia paragraphs it managed about 250 documents per second, which extrapolates to over two hours for the full corpus.

The rewrite computes the entire signature in two array operations. `np.outer(hv, _A)` builds the whole (shingles $\times$ 64) matrix of permuted values at once, the modulo and mask apply elementwise, and `.min(axis=0)` collapses it to the 64-number signature. Measured throughput went to about 2400 documents per second, roughly a 10x speedup.

Why swapping the hash function is legal

This is the insight that makes the rewrite safe. MinHash signatures do not need to match any external standard; they only need to be *internally consistent*. Every document must be hashed the same way as every other, and that is the whole requirement. That freedom permits replacing datasketch's sha1 element hash with the much faster xxhash and writing `hashvalues` directly, as long as all documents share one scheme and one set of permutations. Reusing the seed MinHash's own `_A`, `_B` guarantees exactly that, which is why the LSH banding on top continues to work unchanged.

xxhash is also chosen over Python's builtin `hash()` for a specific reason: `hash()` on strings is salted per process, so it returns different values on different runs and would make the corpus irreproducible.

The README also records a real integration bug: datasketch 2.0 changed the MinHash constructor so that passing `permutations=` without a `scheme` raises `ValueError: scheme must be specified explicitly`. The fix was to read the seed MinHash's scheme (`affine32`) and pass it explicitly everywhere.

A latent trap in `build_minhash`'s signature

`build_minhash(text, num_perm=_NUM_PERM)` takes `num_perm` as a parameter, but `_A` and `_B` are module-level constants built once at `num_perm=64`. Passing anything other than 64 would construct a MinHash claiming a different permutation count while handing it 64 coefficients. Nothing in the repository does this (`Deduper` defaults to 64 and passes its own `self.num_perm`, also 64), so it is not a live bug. But the parameter advertises a flexibility the function does not have. Either wire it through properly or drop the argument.

4.3.4 Deduper: the two filters composed

`Deduper.keep(text)` runs exact first (cheap, short-circuits), then near. It keeps three counters, `n_exact`, `n_near`, `n_kept`, and `stats()` derives the dedup rate from them. Documents are inserted into the LSH under a monotonically increasing integer id, which is never used to look anything up: only `query()`'s truthiness matters, not which document was hit.

The LSH index lives entirely in RAM

`MinHashLSH(threshold=0.8, num_perm=64)` with datasketch's default storage is an in-memory index, and `seen_exact` is an in-memory set. For 2.5 million (v1) and 13.3 million (v2) kept documents, both grow with the corpus. This is the real memory ceiling of the whole pipeline, and it is the one place where the otherwise-strict streaming discipline does not hold. The README names this itself under "What I'd do differently" and proposes persisting signatures to disk so the stage becomes independently resumable and the Jaccard threshold could be retuned without re-hashing. The fix is not implemented; the honest position is that the v2 run's memory footprint worked on the machine it was run on.

4.4 `common.py`: the plumbing that makes stages resumable

Three small pieces, each doing exactly one job.

- `iter_jsonl(path)` yields one parsed object per non-blank line. Streaming, naturally.
- `ShardWriter` writes records to `prefix-00000.jsonl`, `prefix-00001.jsonl`, rolling to a new file every `docs_per_shard` records. It opens lazily (no empty file if nothing is ever written) and supports the `with` statement. `ensure_ascii=False` keeps Urdu readable in the shard rather than escaped to `\u06cc` noise.

- **Stage** implements resume with the simplest mechanism that works: a marker file named `.<stage>.done` that holds the stage's own stats as JSON. `is_done()` is an existence check.

Jargon: JSONL

"JSON Lines": a text file with one complete JSON object per line. Unlike a single JSON array, it can be read one record at a time and appended to without rewriting, which is why every streaming pipeline uses it.

The resume marker is coarser than it looks

The marker is written only after a stage *completes*. If extraction dies 90% through a nine-hour CC-100 pass, no marker exists, and the next run redoes all of it. "Resumable" here means "skip stages that finished", not "continue from where it broke". That is still valuable (the three stages are hours apart) but it is a weaker guarantee than the word usually implies, and worth stating plainly rather than letting an interviewer discover it.

4.5 `build_corpus.py`: the orchestrator

`main()` wires three stage functions in order and writes `data/corpus_stats.json` at the end. The design is worth noting: every stage is a plain function taking `(args, work)`, checks its own marker, and returns its stats dict. There is no framework.

- **extract**: for each configured source, stream documents, `normalize()` the text, split into paragraphs on newlines, and write those that pass `keep_line()` to a per-source shard. Note the granularity change here: a source yields *documents* (whole articles) but the pipeline stores *paragraphs*. That is why 1.73M Wikipedia articles produce 2.46M kept paragraphs.
- **dedup**: read every extract shard through one `Deduper`, writing survivors to `clean/`. Deduplication is corpus-wide, not per-source, so a paragraph appearing in both Wikipedia and CC-100 is caught.
- **split**: read the clean shards and send each record to val when `rng.random()` falls below `args.val_frac` (default 0.01), and to train otherwise. The generator is seeded at 1337, so the split is reproducible.

The split is a per-document coin flip, not a guaranteed 1%

`stage_split` draws an independent random number per document. The expected val share is exactly 1%, and at 2.5 million documents the variance is negligible (the observed v1 split, 25,106 val of 2,515,082, is 0.998%). It is deterministic given the seed, which is what matters for reproducibility. But it is a Bernoulli draw, not a partition, so the count is not exactly 1% and "deterministic 1% holdout" is a description of the mechanism's expectation rather than its guarantee. This is a nitpick, not a defect.

4.6 What the pipeline actually produced

Figure 5 traces the v1 corpus, whose real numbers are the committed `data/corpus_stats.json`. Every figure below was read out of that file, not out of the README.



Figure 5: The v1 corpus funnel. All figures are from the committed `data/corpus_stats.json`, verified by reading it during the writing of this document.

The committed corpus stats describe v1, but the shipped model is v2

`data/corpus_stats.json` contains exactly three sources (wikipedia, leipzig 2016, leipzig 2011) and the v1 numbers above. There is **no** committed stats file for the v2 corpus, the one that CC-100 was added to and that `runs/base_v2` (the project’s headline model) was actually trained on. The v2 table in the README (11.0M CC-100 documents kept, 37.5% dedup rate, 13.3M train docs) is therefore the *only* record of that corpus, and unlike the v1 numbers it cannot be checked against a committed artifact. It is not doubted here; the point is that it rests on the README’s word where v1’s rests on a file. Re-running the pipeline would regenerate the file and overwrite the v1 stats, which may be exactly why it was not re-run.

5 Part IV: The tokenizer

This is the part of the project that most directly answers "why build this at all instead of fine-tuning something". Section 2.3 argued from first principles that an English tokenizer must handle Urdu badly. This part measures it.

5.1 train_tokenizer.py: 56 lines, several real decisions

```

1 tok = Tokenizer(models.BPE(unk_token="<|unk|>"))
2 tok.pre_tokenizer = pre_tokenizers.ByteLevel(add_prefix_space=False)
3 tok.decoder = decoders.ByteLevel()
4 trainer = trainers.BpeTrainer(
5     vocab_size=32000,
6     min_frequency=2,
7     special_tokens=["<|endoftext|>", "<|pad|>", "<|unk|>"],
8     initial_alphabet=pre_tokenizers.ByteLevel.alphabet(),
9     show_progress=True,
10 )
11 tok.train_from_iterator(corpus_lines([data / "train"]), trainer=trainer)
12 tok.post_processor = processors.ByteLevel(trim_offsets=False)

```

Listing 4: The whole configuration of the tokenizer

Unpacking each choice:

Jargon: Byte-level BPE

BPE whose base alphabet is the 256 possible byte values, not the set of characters seen in training. Since every possible input encodes to bytes, and every byte is in the vocabulary, **no input can ever be out-of-vocabulary**. A character-level BPE would have to emit an unknown token for a character it never saw; a byte-level one always has a fallback, at worst spending one token per byte.

- `initial_alphabet=ByteLevel.alphabet()` is what guarantees that property. It seeds the vocabulary with all 256 bytes *before* training, so even bytes that never appear in Urdu text (an emoji, a Cyrillic letter) still have an id. Without it, only bytes observed in the corpus would be present.
- `min_frequency=2` refuses to spend a vocabulary slot on a merge seen once.
- `add_prefix_space=False` is a deliberate departure from GPT-2's default. GPT-2 prepends a space to the input so that "word" and " word" tokenize alike. That convention is built around English's space-delimited orthography; this tokenizer declines it.
- **The tokenizer is trained on data/train only.** `shard_dirs = [data / "train"]`, with the `val` directory pointedly excluded. This matters more than it looks: a tokenizer trained on the validation set would have merges shaped by text the model is later evaluated on, a subtle leak. Getting this right is a mark of care.

Jargon: Special tokens

Ids reserved for control rather than text. `<|endoftext|>` marks a document boundary and is appended after every document during tokenization. `<|pad|>` would fill short sequences to equal length, and `<|unk|>` would represent an unknown symbol.

Two of the three special tokens are never used

`<|pad|>` is declared and given a vocabulary slot, but nothing in the repository ever pads: `TokenLoader` slices fixed-length blocks out of one continuous token stream, so every sequence is exactly `block_size` long by construction and padding is structurally unnecessary. `<|unk|>` is passed to `models.BPE(unk_token=...)` but a byte-level BPE with a full 256-byte initial alphabet can never emit it, by the very design described above. Only `<|endoftext|>` does real work. This costs two vocabulary slots out of 32,000, which is nothing, and both are conventional insurance. It is worth knowing they are inert rather than being asked in an interview what `<|pad|>` is for and discovering the answer is "nothing".

5.2 tokenize_corpus.py: text to a flat array of numbers

Once the tokenizer is frozen, the corpus is encoded once, ahead of time, into a single flat binary file per split. Nothing is tokenized during training.

```

1 for rec in iter_jsonl(shard):
2     src = rec.get("source")
3     if only and src not in only:          # --filter-source
4         continue
5     ids = tok.encode(rec["text"]).ids
6     ids.append(eot_id)                    # every doc ends with <|endoftext|>
7     reps = upsample.get(src, 1) if upsample else 1
8     buf.extend(ids * reps)                # --upsample wikipedia:3
9 arr = np.array(buf, dtype=np.uint16)
10 arr.tofile(fh)
```

Listing 5: The core of `encode_split`

Jargon: uint16

An unsigned 16-bit integer: whole numbers from 0 to 65,535. The vocabulary is 32,000, so every token id fits, and each costs 2 bytes instead of the 4 an `int32` would take. On the 862M-token v2 corpus that is the difference between a 1.7GB file and a 3.4GB one. The choice is only safe because the vocabulary is under 65,536, which is a constraint worth knowing about before someone raises `vocab_size`.

Jargon: Memory-mapping

Handing the operating system a file and asking it to make the bytes appear as if they were an array in memory, without reading them in. Pages load on demand and the OS evicts them under pressure. `np.memmap(bin_path, dtype=np.uint16, mode="r")` is how a 1.7GB corpus is trained on without a 1.7GB allocation.

5.2.1 Upsampling and filtering, and why both exist

The v2 corpus is dominated by CC-100, a web crawl, by sheer volume. Wikipedia is smaller but cleaner. Left alone, the model would spend most of its training reading crawl text. Two flags address this:

- `--upsample wikipedia:3` repeats every Wikipedia document three times in `train.bin`, so a smaller high-quality source is not swamped. This produced the 862.4M-token main mix.
- `--filter-source wikipedia` writes a Wikipedia-only stream, used for a short *annealing* phase at the end of training. This produced the 70.8M-token `data_anneal/train.bin`.

Annealing, in this project's sense

Training first on the big mixed corpus, then finishing on a small high-quality slice. The idea is that the final gradient steps have outsized influence on the model's habits, so ending on clean encyclopedic prose biases the finished model toward it. The v2 run did exactly this, and Section 9 shows the switch is plainly visible in the loss log.

Validation is never upsampled or filtered

`upsample=upsample if train else None` and `only=only if train else None`. The val split is always the full, unweighted distribution. If val were upsampled to match train, validation loss would measure performance on a distorted distribution and would not be comparable across runs with different mixes. This is quietly correct, and it is also why `data_anneal/val.bin` and `data/val.bin` both hold the same 7.3M tokens.

Upsampling repeats a document's tokens back-to-back inside one context

`buf.extend(ids * reps)` is Python list repetition, which lays the three copies down **contiguously**: `[doc, doc, doc]` in the stream, not three copies scattered across the corpus. `TokenLoader` then slices blocks of 1024 tokens at fixed offsets from that stream. A typical kept Wikipedia paragraph is a few hundred characters, so at the measured 4.078 chars/token it is well under 100 tokens, and all three copies of it comfortably fit inside a single 1024-token training block.

The consequence is that the model repeatedly sees a context in which the literal correct continuation is a verbatim copy of text 70 tokens earlier. That is a strong, easily learned signal, and it rewards copying rather than modeling. This is a different thing from what upsampling is meant to do, which is to make Wikipedia's *distribution* count for more across the corpus.

The fix is cheap: repeat at the shard or document-list level, or interleave the copies, rather than repeating the id list in place. Whether it measurably hurt the v2 model is not established here and cannot be established without a retrain. It is stated as a real design defect that survived into the run that produced the headline model, and it is probably the single most substantive code finding in this document.

5.3 The 5.23x claim, verified

The README's headline claim is that GPT-2's tokenizer spends 5.23x more tokens on the same Urdu text. This was checked by running the code rather than trusting the table.

`eval/tokenizer_compare.py` builds a sample by streaming records out of `data/val` until it has `--max-chars` characters, encodes it with both tokenizers, and reports chars-per-token for each. The comparison is deterministic: no sampling, no seed, so it must reproduce exactly.

Verified on this machine, 2026-07-16

A real GPT-2 `tokenizer.json` was found in the local Hugging Face cache and the comparison was run against it. Every number in the README's table reproduced **exactly**, to the digit:

```
1 $ python -m eval.tokenizer_compare --gpt2 <cached gpt2 tokenizer.json> \
2   --max-chars 800000
3 sample chars: 806495
4 ours (32k Urdu BPE): 197758 tokens, 4.078 chars/token
5 gpt2: 1034831 tokens, 0.779 chars/token
6 compression gain: 5.23x fewer tokens with our tokenizer
```

Listing 6: Real output, reproducing the README's table exactly

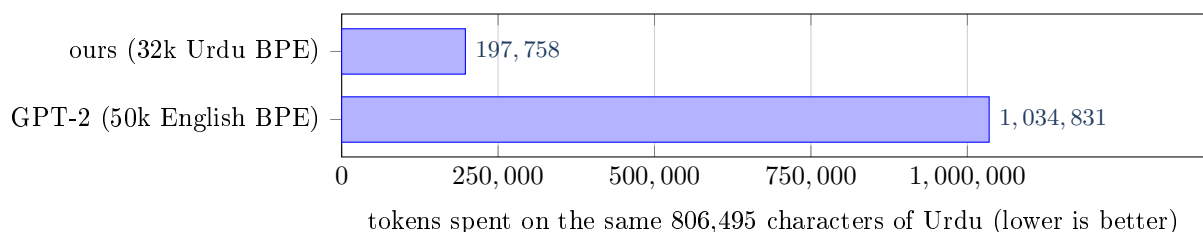


Figure 6: The measured compression gap, reproduced on this machine. GPT-2 needs 5.23x more tokens for identical text. At 0.779 chars/token it is emitting more than one token per Urdu character, consistent with the two-bytes-per-character analysis in Section 2.3.

5.3.1 What the number means in money and context

The claim is not an abstraction. Three concrete consequences follow. All three are **arithmetic on the measured ratio**, not separate measurements, and are labelled as such:

- **Training cost.** The v2 run processed 2.5B tokens for about \$13. The same text through GPT-2's tokenizer would be roughly 13B tokens, so roughly \$68 of GPU time for the same corpus, before any consideration of quality.
- **Context window.** 1024 tokens holds about 4,176 Urdu characters with this tokenizer, and about 798 with GPT-2's. The same model architecture would see about a fifth as much Urdu at once.
- **Learnability.** A model whose tokens are mostly single bytes must spend its early capacity learning to reassemble characters from bytes, before it can learn any Urdu. An Urdu-native tokenizer hands it meaningful units on the first step.

The README's stated command does not reproduce the README's number

The README documents the measurement as `python -m eval.tokenizer_compare --gpt2 <gpt2 tokenizer.json>`, with no `--max-chars`. But `--max-chars` defaults to **500,000**, and running the command exactly as written yields a 504,251-character sample, 125,634 of our tokens against GPT-2's 645,497, and a gain of **5.14x**, not 5.23x. Reproducing the README's exact figures requires `--max-chars 800000`, which is the flag that produces the cited 806,495-character sample.

Both numbers are real and neither is inflated (5.14x and 5.23x are the same finding). The defect is purely documentary: the command printed next to the table is not the command that produced the table. One flag added to the README closes it. Worth fixing precisely because the number is genuine and reproduces perfectly once the flag is there; it would be a shame to have a reviewer type the documented command, get a different number, and start doubting the rest.

The comparison samples from val, then train, which is fine but undocumented

`sample_text` iterates for `split` in `["val", "train"]`. Since `data/val` alone holds 8.7MB of text, the 800k-character sample never reaches `data/train` in practice, so the sample is entirely held-out data. That is the right thing and it is what "held-out Urdu sample" in the README means. But it is a consequence of the `val` directory happening to be large enough, not an enforced property: with a smaller `val` split the sample would silently start including training text, and the tokenizer would be measured partly on data it was trained on. A single assertion, or dropping `"train"` from that list, would make the guarantee real rather than incidental.

5.3.2 The byte-level fallback path

If `--gpt2` is not supplied, the script compares against a synthetic worst case: one token per UTF-8 byte. The docstring is careful to call this "the worst case GPT-2 approaches on unseen scripts" rather than GPT-2 itself. The measurement above shows how close that approximation is: GPT-2's real 0.779 chars/token against the byte baseline's roughly 0.5 for two-byte characters. GPT-2 does slightly better than pure bytes, because a few merges do fire, but it is in the same regime. The fallback is an honest stand-in, labelled as one.

6 Part V: The model

`model/transformer.py` is 186 lines and contains the entire network. Nothing is imported from the `transformers` library. It is a modern decoder-only stack: not a GPT-2 reimplementation, but the Llama-family design (RMSNorm, RoPE, SwiGLU, no biases, tied embeddings).

6.1 `config.py`: one dataclass, four presets

`ModelConfig` is a `@dataclass` holding every architectural knob. `__post_init__` does two jobs: it expands the `n_kv_head == 0` sentinel to mean "no grouped-query attention", and it asserts the two divisibility constraints the rest of the code depends on (`n_embd` divisible by `n_head`, `n_head` divisible by `n_kv_head`). Failing loudly at construction beats failing with a shape mismatch fifty lines deep in attention.

Verified: every preset's parameter count matches the README

The four presets were instantiated at the real 32k vocabulary and counted during the writing of this document. All four match the README's table exactly.

preset	layers	d_model	heads	ctx	embedding	blocks	total
tiny	6	256	4	512	8.19M	4.82M	13.01M
small	6	512	8	1024	16.38M	19.27M	35.66M
base	14	768	12	1024	24.58M	99.11M	123.69M
medium	24	1024	16	1024	32.77M	303.61M	336.38M

Table 3: The four presets, measured on this machine, not copied from the README. The embedding column is `vocab_size × n_embd`; because embeddings are tied it is counted once, not twice.

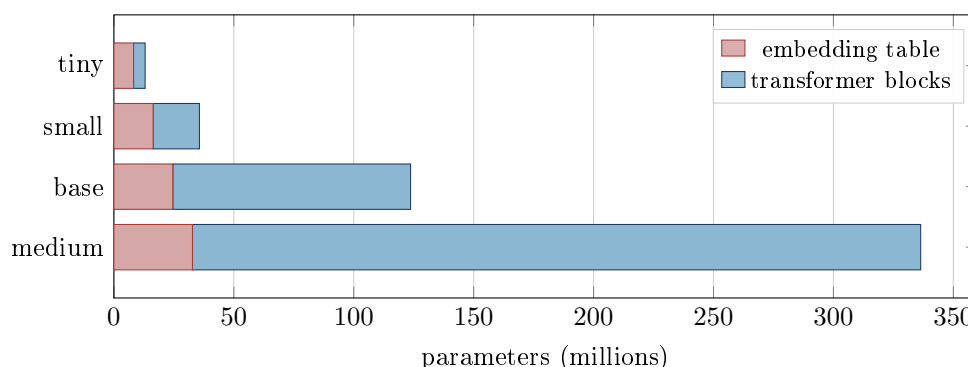


Figure 7: Where the parameters live. For `tiny`, the vocabulary table is 63% of the whole model; by `medium` it is under 10%. This is the point the README makes honestly rather than hiding.

The "10M-parameter model" is a fiction, and the project says so

The `tiny` preset's comment in `configs/tiny.yaml` still says "~10M params", but a 32,000-token vocabulary at `d_model=256` costs 8.19M parameters before a single transformer block exists. The blocks add only 4.82M. So a genuinely tiny transformer still lands at 13.0M, and the initial parameter-count test failed at 15.2M while the author was tuning it.

The choice made was to tune the block dimensions to a defensible 13M and *document* that 8.2M of it is vocabulary, rather than shrink the vocabulary to hit a round headline number. That is the right call: shrinking the vocab to reach "10M" would have degraded the very tokenizer the project exists to demonstrate, in exchange for a prettier number. This is a good

thing to be able to say out loud in an interview.

6.2 The block, drawn

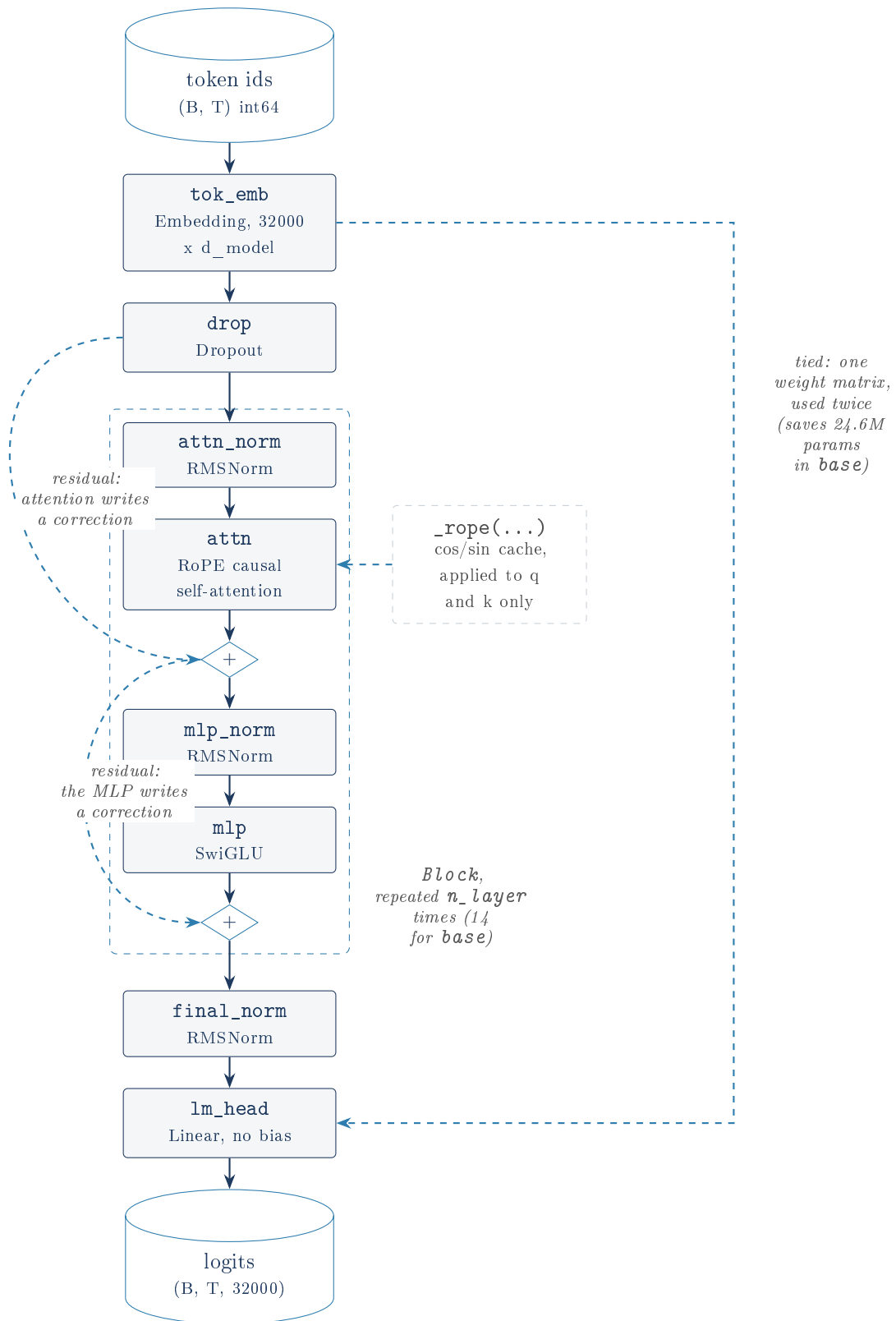


Figure 8: The full forward pass. Dashed arrows are residual connections and the tied weight matrix. Note that both norms sit *before* their sublayer, not after.

Jargon: Residual connection

The `x = x + sublayer(norm(x))` pattern. The sublayer computes a *correction* that is added to its input, rather than replacing it. This gives gradients a clean path straight from the loss back to the earliest layers, which is what makes deep stacks trainable at all. Both `+` nodes in Figure 8 are these.

Jargon: Pre-norm vs post-norm

Where the normalization sits. The original transformer normalized *after* the sublayer (post-norm), which needs careful learning-rate warmup to avoid diverging. Pre-norm, used here, normalizes the sublayer's input and leaves the residual path untouched. It is markedly more stable, and it is why the residual stream in Figure 8 runs from embedding to `final_norm` without ever passing through a normalization.

6.3 RMSNorm

```
1 class RMSNorm(nn.Module):
2     def __init__(self, dim, eps=1e-5):
3         super().__init__()
4         self.eps = eps
5         self.weight = nn.Parameter(torch.ones(dim))
6
7     def forward(self, x):
8         dtype = x.dtype
9         x = x.float() # fp32 for stability
10        x = x * torch.rsqrt(x.pow(2).mean(-1, keepdim=True) + self.eps)
11        return (x * self.weight.float()).to(dtype)
```

Listing 7: RMSNorm in full

Jargon: Normalization, and RMSNorm specifically

As numbers flow through layers their scale drifts, and drift destabilizes training. Normalization rescales each vector to a standard size. The older LayerNorm subtracts the mean and divides by the standard deviation, then applies a learned scale and shift. RMSNorm drops the mean-subtraction and the shift entirely: it divides by the root mean square and applies one learned scale. It works about as well, with fewer operations and fewer parameters, which is why every recent model uses it.

Two details in those five lines are load-bearing:

- `eps` inside the square root prevents division by zero for an all-zero vector.
- The `x.float()` / `.to(dtype)` sandwich is the important one. Training runs in `bfloat16` (Section 7.4), which has very few mantissa bits. Squaring and averaging in `bf16` loses precision exactly where it matters, so the statistic is computed in `fp32` and only the result is cast back. This is a standard, deliberate mixed-precision idiom, not an accident.

6.4 Rotary position embeddings (RoPE)

Jargon: Positional encoding, and why it is needed

Attention is a weighted average, and averages have no order: without help, "dog bites man" and "man bites dog" look identical to it. Something must inject position. GPT-2 added a learned position vector to each token's embedding. RoPE takes a different route.

Jargon: RoPE

Instead of *adding* position information, RoPE *rotates* each query and key vector by an angle proportional to its position. Pairs of dimensions are treated as points on a plane and spun. The elegance is what happens next: when attention takes the dot product of a query at position m with a key at position n , the rotations combine so the result depends only on the *difference* $m - n$. Relative position falls out of the arithmetic for free, with zero parameters.

```

1 def build_rope_cache(seq_len, head_dim, theta, device, dtype):
2     inv_freq = 1.0 / (theta ** (torch.arange(0, head_dim, 2).float() / head_dim))
3     t = torch.arange(seq_len).float()
4     freqs = torch.outer(t, inv_freq) # (seq_len, head_dim/2)
5     return freqs.cos().to(dtype), freqs.sin().to(dtype)
6
7 def apply_rope(x, cos, sin):          # x: (B, n_head, T, head_dim)
8     T = x.shape[-2]
9     cos = cos[:T].unsqueeze(0).unsqueeze(0)
10    sin = sin[:T].unsqueeze(0).unsqueeze(0)
11    x1, x2 = x.chunk(2, dim=-1)
12    rot = torch.cat((-x2, x1), dim=-1) # the 90-degree rotation
13    cos_full = torch.cat((cos, cos), dim=-1)
14    sin_full = torch.cat((sin, sin), dim=-1)
15    return x * cos_full + rot * sin_full

```

Listing 8: Building and applying the rotation

`inv_freq` builds a spectrum of rotation speeds from `theta` (10,000 here): the first dimension pair rotates fast and distinguishes neighbours, the last rotates extremely slowly and distinguishes distant positions. Together they encode position at every scale. `rot = cat((-x2, x1))` is the standard trick for rotating every pair at once: it is what $(a, b) \rightarrow (-b, a)$ does, vectorized across the whole tensor.

RoPE is applied to queries and keys only, never to values

`q = apply_rope(q, cos, sin)` and `k = apply_rope(k, cos, sin)`, but `v` is untouched. This is correct and it is the whole design: position must influence *who attends to whom* (which is decided by the query-key dot product), but the content that gets passed along once a position is selected should not be scrambled by where it happened to sit. Getting this wrong is a classic RoPE bug; this code gets it right.

The RoPE cache key is subtly wasteful

`_rope(self, T, device, dtype)` keys its cache on `(T, device, dtype)`, but the value it builds ignores `T` entirely and always constructs the cache at `self.cfg.max_seq_len`. So a forward pass at `T=64` and one at `T=128` create two cache entries holding **identical** tensors. Nothing is incorrect (`apply_rope` slices `cos[:T]`, so any entry long enough works), and training always uses one fixed `T` so exactly one entry is ever built. But `generate()` calls the model with a growing `T` on every single step, so sampling 60 tokens builds up to 60 identical full-size caches in a plain dict that is never evicted. For the `base` model that is 60 copies of a 1024x32 tensor: small in absolute terms, but pointless. Keying on `(device, dtype)` alone would be strictly better and is a one-line change.

A related note: `_rope_cache` is a plain Python dict, not a registered buffer, so it is invisible to `.to()`, `state_dict()` and `.cuda()`. That is actually *why* the device is in the key, and it means checkpoints do not carry redundant cache tensors. Defensible, but it is the kind of thing that looks like an oversight until you work out why it is there.

6.5 Attention

```

1 q = self.q_proj(x).view(B, T, self.n_head, self.head_dim).transpose(1, 2)
2 k = self.k_proj(x).view(B, T, self.n_kv_head, self.head_dim).transpose(1, 2)
3 v = self.v_proj(x).view(B, T, self.n_kv_head, self.head_dim).transpose(1, 2)
4
5 q = apply_rope(q, cos, sin)
6 k = apply_rope(k, cos, sin)
7
8 if self.n_kv_head != self.n_head:                # grouped-query attention
9     rep = self.n_head // self.n_kv_head
10    k = k.repeat_interleave(rep, dim=1)
11    v = v.repeat_interleave(rep, dim=1)
12
13 y = F.scaled_dot_product_attention(q, k, v, is_causal=True,
14                                   dropout_p=self.dropout if self.training else 0.0)
15 y = y.transpose(1, 2).contiguous().view(B, T, self.n_head * self.head_dim)
16 return self.o_proj(y)

```

Listing 9: The attention forward pass

Jargon: Multi-head attention

Rather than one attention computation over 768 dimensions, run 12 independent ones over 64 dimensions each and concatenate. Each "head" can specialize: one might track subject-verb agreement, another long-range topic. The `view(...).transpose(1,2)` pair is what reshapes (B, T, 768) into (B, 12, T, 64) so the heads compute in parallel as a batch dimension.

Jargon: Grouped-query attention (GQA)

An inference optimization. Generation caches every past key and value, and that cache dominates memory. GQA gives each head its own query but shares keys and values across groups of heads, shrinking the cache proportionally. Here `n_kv_head` controls it, and `repeat_interleave` expands the shared keys back out to per-head shape.

GQA is implemented but not used by any preset, and its implementation is the naive one

Every one of the four presets leaves `n_kv_head` at its default 0, which `__post_init__` expands to `n_head`, which makes `n_kv_head != n_head` false and skips the branch entirely. So no shipped model uses GQA; the README's "grouped-query-capable" is exactly the right word, and the code is honest about it.

Two things are worth knowing anyway. First, the `repeat_interleave` implementation materializes the expanded keys and values in full, which means it saves *no memory at all* during the forward pass. It is a correctness shim, not an optimization. Real GQA savings come from the KV cache during generation, and this model has no KV cache (see below), so there is currently nothing for GQA to save. Second, that is fine: the branch is there so the config surface is honest and a future run can turn it on. It just should not be described as a performance feature, because as implemented it is not one.

One call does the mask, the scaling and the Flash kernel

`F.scaled_dot_product_attention(q, k, v, is_causal=True)` is doing four things that older implementations spell out by hand: it scales by $1/\sqrt{d_{\text{head}}}$, builds and applies the causal mask, softmaxes, and dispatches to a fused FlashAttention kernel on CUDA when the shapes allow. `is_causal=True` means the triangular mask in Figure 1 is never materialized as a tensor at

all, which is where a large part of the memory saving comes from. The README's claim that "RoPE plus `scaled_dot_product_attention(is_causal=True)` is enough to get a correct, Flash-ready causal transformer with no hand-written mask" is accurate.

Jargon: FlashAttention

An attention implementation that never writes the full ($T \times T$) attention matrix to GPU memory, computing it in tiles inside fast on-chip memory instead. It is exact, not an approximation, and it is the reason a 1024-token context is cheap. It is what delivered the measured $\sim 105,000$ tokens/second in Section 9.

`bias=False` on all four projections is another Llama-family choice: biases contribute little in transformers and cost parameters and a kernel launch. `contiguous()` before the final `view` is required because `transpose` returns a non-contiguous tensor that `view` cannot reshape.

6.6 SwiGLU

```

1 hidden = int(cfg.mlp_ratio * cfg.n_embd)      # mlp_ratio = 8/3
2 hidden = 64 * ((hidden + 63) // 64)          # round up to a multiple of 64
3 self.gate_proj = nn.Linear(cfg.n_embd, hidden, bias=False)
4 self.up_proj   = nn.Linear(cfg.n_embd, hidden, bias=False)
5 self.down_proj = nn.Linear(hidden, cfg.n_embd, bias=False)
6
7 def forward(self, x):
8     return self.down_proj(F.silu(self.gate_proj(x)) * self.up_proj(x))

```

Listing 10: The MLP

Jargon: MLP / feed-forward block

Attention moves information between positions; the MLP does the thinking at each position independently. Classically it projects up to 4x the width, applies a non-linear function, and projects back down.

Jargon: SwiGLU, and the gating idea

A gated variant. The input is projected *twice*, into a "gate" and an "up" path. The gate goes through SiLU ($x \cdot \sigma(x)$, a smooth version of ReLU) and then **multiplies** the up path elementwise. So the gate learns, per dimension, how much of the up path to let through. That multiplicative interaction is more expressive than a single projection, and SwiGLU consistently outperforms plain GELU at equal parameter count.

Why the ratio is 8/3, and why base needs 14 layers rather than 12

SwiGLU needs **three** weight matrices where a classic MLP needs two. At the textbook 4x width that would be 50% more parameters than a GELU block. The 8/3 ratio exists to cancel exactly that: $3 \times \frac{8}{3} = 8 = 2 \times 4$, so a SwiGLU block at 8/3 has the same parameter count as a GELU block at 4x.

The consequence is the one the README calls out. Each **base** layer is parameter-cheaper than a GPT-2 layer of the same width, so the textbook 12 layers do not reach 124M. The preset uses **14**. Depth, not the memorized layer count, is what hits a target size. This is a genuinely non-obvious thing to have worked out and it is a strong interview answer.

The rounding to a multiple of 64 is a GPU alignment concern: tensor-core matrix multiplies are fastest on dimensions that are multiples of 64, so the ratio is treated as a target rather than an exact requirement.

6.7 UrduLM: assembly, tying, and initialization

6.7.1 Weight tying

```
1 self.lm_head = nn.Linear(cfg.n_embd, cfg.vocab_size, bias=False)
2 if cfg.tie_embeddings:
3     self.lm_head.weight = self.tok_emb.weight
```

Listing 11: Tying, in two lines

Jargon: Weight tying

Using one matrix for both the input embedding (id to vector) and the output projection (vector to scores over ids). They are natural transposes of each other: both encode "what does token i look like in vector space". Sharing them saves $\text{vocab_size} \times \text{n_embd}$ parameters (24.6M for `base`) and usually improves quality, because one matrix gets gradient signal from both directions.

This is a genuine assignment of the same Python object, not a copy, so the two stay identical for the whole of training. `tests/test_model.py` asserts it at the level that actually proves it: `model.lm_head.weight.data_ptr() == model.tok_emb.weight.data_ptr()`, comparing the memory addresses rather than the values. Comparing values would pass even if they were two matrices that merely started equal and then diverged. That is a well-designed test.

6.7.2 Initialization

```
1 self.apply(self._init_weights) # everything: normal(0, 0.02)
2 for name, p in self.named_parameters(): # then override the residual outputs
3     if name.endswith("o_proj.weight") or name.endswith("down_proj.weight"):
4         nn.init.normal_(p, mean=0.0, std=0.02 / math.sqrt(2 * cfg.n_layer))
```

Listing 12: The two-stage init

Why the residual projections get a smaller initial scale

Every block *adds* to the residual stream. With L blocks each contributing independent noise, the stream's variance grows roughly linearly with depth, so a deep stack starts with a wildly-scaled output before it has learned anything. Scaling the two projections that write into the residual stream (`o_proj` from attention, `down_proj` from the MLP) by $1/\sqrt{2L}$ keeps the accumulated variance roughly constant regardless of depth. The factor 2 is because each layer writes twice, once from attention and once from the MLP. This is the GPT-2 initialization scheme, and it is correctly reproduced here.

6.7.3 forward

The forward pass asserts `T <= cfg.max_seq_len` (a real guard: RoPE's cache is built to exactly that length), embeds, runs the blocks, normalizes, projects to logits, and optionally computes the loss with `ignore_index=-100`.

Jargon: ignore_index=-100

PyTorch's convention for "do not score this position". Any target set to -100 contributes nothing to the loss. Pretraining never uses it (every position is a real target), but `eval/cloze.py` depends on it entirely: it masks out the context so only the candidate answer's tokens are scored. See Section 8.3.

6.7.4 generate

```

1 for _ in range(max_new_tokens):
2     idx_cond = idx[:, -self.cfg.max_seq_len:]          # crop to the context window
3     logits, _ = self(idx_cond)
4     logits = logits[:, -1, :] / max(temperature, 1e-6)
5     if top_k is not None:
6         v, _ = torch.topk(logits, min(top_k, logits.size(-1)))
7         logits[logits < v[:, [-1]]] = -float("inf")
8     probs = F.softmax(logits, dim=-1)
9     next_id = torch.multinomial(probs, num_samples=1)
10    idx = torch.cat((idx, next_id), dim=1)

```

Listing 13: Autoregressive sampling**Jargon: Temperature**

A divisor on the logits before softmax. Below 1 it sharpens the distribution (more predictable, more repetitive); above 1 it flattens it (more varied, more likely to be nonsense). `max(temperature, 1e-6)` guards against a division by zero when someone asks for greedy decoding by passing 0.

Jargon: Top-k sampling

Keep only the k highest-scoring tokens and set the rest to negative infinity, which softmax turns into exactly zero probability. This prevents the long tail of thousands of barely-plausible tokens from collectively stealing a meaningful share of the draws. The project samples at `top_k=40`.

generate has no KV cache, so sampling is quadratic

Every iteration re-runs the *entire* forward pass over the whole prompt so far, and then throws away all of it except the last position's logits. Generating n tokens therefore costs $O(n^2)$ work instead of the $O(n)$ that caching past keys and values would give. Standard implementations keep a KV cache exactly for this.

For this project's actual usage (a handful of 60 to 80 token samples for the script-mix diagnostic) the waste is invisible and the simplicity is worth more. But it means this model is not deployable for interactive serving as written, and it is the reason the GQA branch above has nothing to optimize: GQA shrinks a KV cache, and there is no KV cache. Naming this before an interviewer does is much better than the reverse.

num_params(non_embedding=True) has two branches that do the same thing

```

1 n = sum(p.numel() for p in self.parameters())
2 if non_embedding and not self.cfg.tie_embeddings:
3     n -= self.tok_emb.weight.numel()
4 if non_embedding and self.cfg.tie_embeddings:
5     n -= self.tok_emb.weight.numel()

```

The two conditions are exhaustive and mutually exclusive, and their bodies are identical, so the whole thing is equivalent to a single `if non_embedding:`. It reads as though it were meant to handle the two cases differently (in the untied case `sum(parameters())` counts both the embedding and `lm_head`, so subtracting only `tok_emb` still leaves the head's copy in the total) and then collapsed. The result is not wrong for any shipped preset, since all four tie embeddings and nothing calls the flag anyway. It is dead branching that should be one line, and if the untied case is ever used the "non-embedding" count will silently include the output head.

6.8 The tests

Verified: all 13 tests pass on this machine

`python -m pytest tests/ -q` was run during the writing of this document: **13 passed in 55.42s**, matching the README's claim of 13 tests (5 model, 8 pipeline).

The best of them is `test_causal_masking`, because it tests the *property* rather than the implementation:

```
1 base, _ = model(x)
2 x2 = x.clone()
3 x2[0, -1] = (x2[0, -1] + 1) % cfg.vocab_size      # perturb the LAST token
4 pert, _ = model(x2)
5 assert torch.allclose(base[:, :-1], pert[:, :-1], atol=1e-5)      # earlier: unchanged
6 assert not torch.allclose(base[:, -1], pert[:, -1], atol=1e-5)    # last: changed
```

Listing 14: Testing causality by experiment

Change a future token; if any earlier position's output moves, information flowed backwards and the mask is broken. The second assertion is what makes it a real test rather than a tautology: without it, a model that ignored its input entirely would pass. This would catch a swapped mask, a missing `is_causal`, or an off-by-one in RoPE, none of which a shape test would notice.

`test_preset_param_counts` asserts each preset lands inside a band (for instance `base` in 115M to 135M). Bands rather than exact numbers is the right choice: it pins the intent without making every dimension tweak a test failure.

7 Part VI: Training

`train.py` is 268 lines and is the piece that had to work unattended on a metered GPU. Its design priorities show that: budget in tokens rather than steps, resume exactly, log everything, and never surprise the operator.

7.1 TokenLoader: batches from a memory-mapped stream

```
1 self.data = np.memmap(bin_path, dtype=np.uint16, mode="r")
2 n_blocks = (len(self.data) - 1) // block_size
3 self.starts = np.arange(n_blocks) * block_size
4 rng = np.random.default_rng(seed)
5 rng.shuffle(self.starts)          # fixed shuffle, seeded
6 self.cursor = 0
```

Listing 15: Setting up the sampler

The whole corpus is one long array of token ids. The loader precomputes every valid block start, shuffles that list once with a seeded generator, and walks it with a cursor.

Why the cursor is the entire resume mechanism

The shuffle is deterministic given the seed, so `self.starts` is the *same list in the same order* on every run of every process. That means the loader's complete state is one integer: how far down the list it has walked. `state_dict()` returns `{"cursor": self.cursor}` and nothing else. Restore that integer and the exact same batch sequence resumes. This is a genuinely elegant reduction, and it is why resume here is trustworthy rather than approximate.

```
1 idxs = []
2 need = self.batch_size * self.world_size
3 while len(idxs) < need:
4     if self.cursor >= len(self.starts):
5         self.cursor = 0          # epoch wrap
6         idxs.append(self.starts[self.cursor])
7         self.cursor += 1
8 idxs = idxs[self.rank::self.world_size][: self.batch_size]
9 x = np.stack([self.data[i : i + self.block_size].astype(np.int64) for i in idxs])
10 y = np.stack([self.data[i + 1 : i + 1 + self.block_size].astype(np.int64) for i in idxs])
```

Listing 16: Drawing one batch

x and y are the same slice, offset by one

This is the whole of next-token prediction, in two lines. `y` is `x` shifted left by one position, so the target at every position is simply the next token. Combined with causal masking, one forward pass over a 1024-token block trains 1024 separate predictions at once. That density is why pretraining works at all.

Every rank draws the *same need* indices and then takes its own stride (`idxs[rank::world_size]`). This is deliberate: since all ranks advance the cursor identically, the cursor stays synchronized across processes without any communication, and rank 0's saved cursor is valid for the whole job.

Three real limitations of the loader

- **The epoch wrap does not reshuffle.** On `cursor = 0` the same order repeats. The v2 run made roughly 3 passes over its data, so it saw the identical block sequence three times. Reshuffling with a per-epoch seed is standard and is strictly better; the cost of not doing it is small but real.

- **Blocks are cut at fixed stride offsets.** Starts are always multiples of `block_size`, so the corpus is carved into a fixed tiling and no block ever straddles a boundary differently between epochs. A random-offset scheme would give more distinct contexts from the same data. Again small, again free.
- **No DataLoader, no prefetch thread.** Batches are built synchronously in the training loop, so the GPU idles during the numpy slicing and the host-to-device copy. At the measured $\sim 105,000$ tokens/second this evidently was not the bottleneck, so the simplicity was the right trade. It is worth knowing it is a trade.

Documents are not respected: blocks straddle `<|endoftext|>` freely

Nothing aligns block boundaries to document boundaries. A 1024-token block routinely contains the tail of one document, an `<|endoftext|>`, and the head of the next, and the model is asked to predict across that seam using unrelated context. This is exactly what GPT-2 and nanoGPT do and is entirely conventional, so it is not a defect. It is worth being able to say why it is acceptable: the `<|endoftext|>` token is the signal that teaches the model a document ended, and attention learns to discount context before it.

7.2 The optimizer and the schedule

Jargon: Gradient descent, and AdamW

Training computes, for every parameter, the direction that would reduce the loss (the *gradient*), and steps a little way along it. Plain descent uses the raw gradient. Adam keeps running averages of each parameter's recent gradients and their squared magnitudes, and uses them to give every parameter its own effective step size, which is far more robust. AdamW adds *decoupled weight decay*: a gentle pull of every weight toward zero, applied separately from the gradient step, which regularizes without corrupting Adam's statistics.

The configuration is `betas=(0.9, 0.95)` and `weight_decay=0.1`. Both are the GPT-3 paper's values and are conventional for this model class: `beta2=0.95` rather than the PyTorch default 0.999 makes the second-moment estimate adapt faster, which is the standard choice for language models.

```

1 def cosine_lr(step, warmup, total, base_lr, min_lr):
2     if step < warmup:
3         return base_lr * (step + 1) / warmup      # linear warmup
4     if step >= total:
5         return min_lr
6     ratio = (step - warmup) / max(1, total - warmup)
7     coeff = 0.5 * (1 + math.cos(math.pi * ratio)) # 1 -> 0 over the run
8     return min_lr + coeff * (base_lr - min_lr)

```

Listing 17: The learning-rate schedule, in seven lines

Jargon: Learning rate, warmup, and cosine decay

The learning rate is how far each step moves. It is not constant. *Warmup* ramps it up linearly from near zero over the first steps, because a randomly initialized model produces enormous unreliable gradients and a full-size step on them can destroy the model in the first minute. *Cosine decay* then eases it smoothly down to a small floor, so early training explores broadly and late training settles precisely.

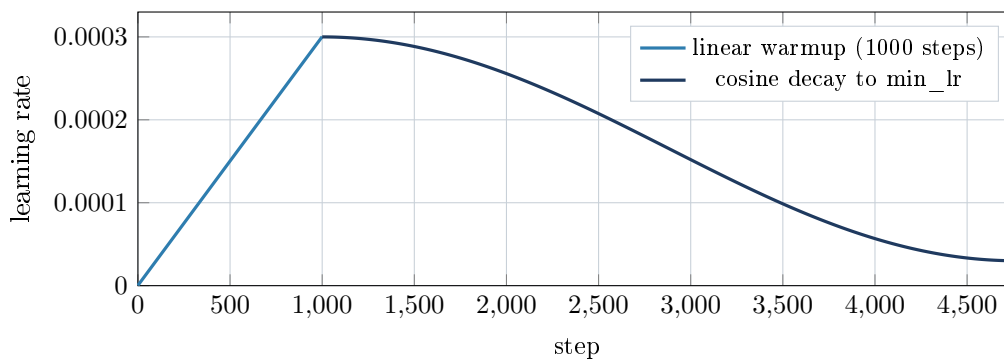


Figure 9: The schedule `cosine_lr` produces for the `base` preset's committed values (`lr 3e-4`, `min_lr 3e-5`, `warmup_steps 1000`), drawn over the v2 run's actual 4,768 steps.

Warmup divides by warmup, not warmup-1, so the peak LR is never reached

`base_lr * (step + 1) / warmup` at `step = warmup - 1` gives exactly `base_lr`, and then the next step takes the cosine branch at `ratio = 0`, which also gives `base_lr`. So the schedule is continuous and correct. This is worth checking rather than assuming, because an off-by-one here is a classic silent bug that either skips the peak or double-counts it. This code gets it right.

7.3 Token-count budgeting

```
1 tokens_per_step = batch_size * block_size * grad_accum * world_size
2 total_steps = max(1, max_tokens // tokens_per_step)
3 warmup_steps = cfg.get("warmup_steps", max(1, int(0.03 * total_steps)))
```

Budget in tokens, derive steps

The config names `max_tokens`, not `max_steps`. Steps are derived. This is the right primitive: "step" is meaningless without knowing the batch size, so a step-budgeted config silently changes the amount of training whenever the batch changes, while a token budget stays comparable across hardware, batch sizes and world sizes. It is also what makes the Chinchilla reasoning in `docs/GPU_RUN.md` directly expressible in the config.

For the `base` preset: $32 \times 1024 \times 16 \times 1 = 524,288$ tokens per step. This is verifiable against the committed log: the v2 run's records step in increments of exactly 10,485,760 tokens per 20 steps, which is $20 \times 524,288$.

7.4 Gradient accumulation, and mixed precision

Jargon: Gradient accumulation

A large batch is statistically desirable but may not fit in VRAM. Accumulation runs `grad_accum` smaller "micro-batches", each doing a backward pass that *adds* into the same gradient buffers, and only then takes one optimizer step. The result is mathematically the same as one large batch, at the cost of more time. Here 16 micro-batches of 32 sequences give an effective batch of 512 sequences, or 524,288 tokens.

```
1 optimizer.zero_grad(set_to_none=True)
2 loss_accum = 0.0
3 for micro in range(grad_accum):
4     x, y = train_loader.next_batch()
5     if is_dist():
```

```

6     model.require_backward_grad_sync = (micro == grad_accum - 1)
7     with ctx:
8         _, loss = model(x, y)
9         loss = loss / grad_accum      # <- so the sum is a mean
10        loss.backward()
11        loss_accum += loss.item()
12    torch.nn.utils.clip_grad_norm_(raw_model.parameters(), grad_clip)
13    optimizer.step()

```

Listing 18: The accumulation loop

Three correct details:

- `loss / grad_accum` is essential. Backward passes *sum* into the gradient buffers, so without the division the effective gradient would be 16x too large and the learning rate 16x too high.
- `require_backward_grad_sync = (micro == grad_accum - 1)` is the DDP optimization that matters most. By default DDP synchronizes gradients across GPUs on *every* backward pass; here it is suppressed for the first 15 micro-batches and allowed only on the last, cutting inter-GPU communication by 16x. Getting this right is the difference between DDP helping and DDP being a bottleneck.
- `set_to_none=True` frees the gradient tensors rather than filling them with zeros: slightly less memory, slightly faster.

Jargon: Gradient clipping

If a batch produces a freak gradient, one step can wreck the model. `clip_grad_norm_` measures the total magnitude of all gradients and, if it exceeds `grad_clip` (1.0 here), scales them all down proportionally. It preserves the direction and caps the size. This is cheap insurance for a long unattended run.

Jargon: bfloat16 and AMP

Automatic Mixed Precision runs most operations in a 16-bit format for speed and memory while keeping the master weights in 32-bit. *bfloat16* keeps float32's full exponent range and sacrifices mantissa bits, which means it cannot overflow where fp32 would not, and therefore needs no loss scaling at all (unlike fp16). It is the reason `torch.cuda.amp.GradScaler` appears nowhere in this file.

```

1    use_bf16 = device == "cuda" and torch.cuda.is_bf16_supported()
2    ctx = (torch.autocast(device_type="cuda", dtype=torch.bfloat16)
3          if use_bf16 else torch.autocast(device_type="cpu", enabled=False))

```

Listing 19: The precision decision

The fallback is an `autocast` context with `enabled=False`, which is a no-op context manager. That is a tidy way to keep one `with ctx:` in the loop instead of branching, and CPU training runs in plain fp32.

7.5 Checkpointing and resume

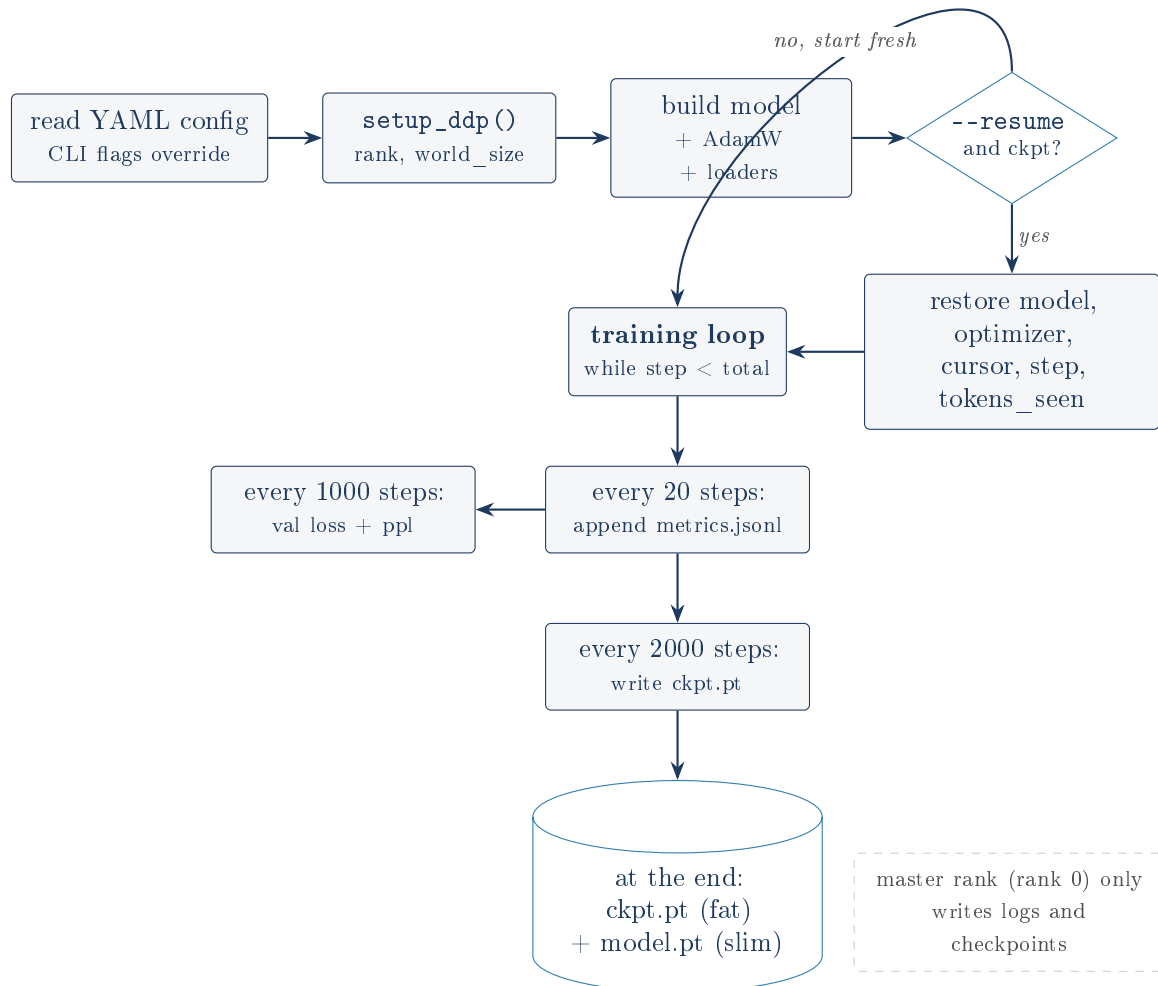


Figure 10: The control flow of `train.py`. Intervals shown are the `base` preset’s committed values.

Two checkpoints are written, and the distinction is deliberate:

	<code>ckpt.pt</code> (fat)	<code>model.pt</code> (slim)
contents	model, optimizer, data cursor, step, tokens_seen, config	model, config, step, tokens_seen
size (base)	1.48GB	495MB
purpose	resume a training run	inference and release

Table 4: The two checkpoint formats. The optimizer’s AdamW state is two extra numbers per parameter, which is why the fat checkpoint is roughly 3x the slim one.

Both files carry config, which is what makes eval scripts work

`torch.save({... , "config": model_cfg.__dict__})` means a checkpoint is self-describing. `eval/_shared.py`’s `load_model` does `ModelConfig(**ck["config"])` and rebuilds the exact architecture from the file alone, with no need to know which preset or which config file produced it. This is why `python -m eval.perplexity --ckpt runs/tiny/model.pt` just works.

The docstring promises RNG restoration that the checkpoint does not contain

`train.py`'s module docstring advertises: "full resume: model + optimizer + scheduler + data cursor + **RNG**". The checkpoint's actual keys are `model`, `optimizer`, `data`, `step`, `tokens_seen`, `config`. **No RNG state is saved or restored.**

How much does it matter? Less than it sounds. The data order is governed by the seeded shuffle and the cursor, both of which *are* restored, and dropout is 0.0 in every committed config, so the only consumer of the RNG in the training path is the initialization that a resume overwrites anyway. So the resume is very likely bit-reproducible in practice despite the missing state. The defect is the claim, not the behaviour. The README's own wording ("optimizer + scheduler + data cursor") is accurate; only the docstring overreaches. Fix the docstring.

Note also that "scheduler" is restored in a subtle sense: there is no `torch.optim.lr_scheduler` object at all. The LR is recomputed from `step` by `cosine_lr` every iteration, so restoring `step` restores the schedule exactly. That is a better design than a stateful scheduler, and it is worth pointing out rather than letting it look like an omission.

A crash between checkpoint intervals loses up to 2000 steps

`ckpt_interval` is 2000 for `base`, which at 524,288 tokens per step is over 1B tokens, roughly 2.8 hours of A100 time at the measured throughput. A crash at step 3,999 would resume from step 2,000. Given the v2 run cost \$13.06 in total, the exposure is a few dollars, so the setting is defensible. But the number was evidently chosen for disk-write frequency rather than by reasoning about the loss window, and 500 would have cost almost nothing extra.

7.6 The configs

	block	batch	accum	tokens/step	max_tokens	lr
tiny	256	16	2	8,192	4.096M	6e-4
small	1024	24	8	196,608	2B	6e-4
base	1024	32	16	524,288	450M	3e-4
medium	1024	32	16	524,288	3.45B	2.5e-4

Table 5: The four committed configs. Learning rate falls as model size rises, which is the conventional relationship.

`configs/base.yaml` does not reproduce `runs/base_v2`, and nothing in the repo records what would

This is the most consequential documentation gap in the project.

The committed `configs/base.yaml` says `max_tokens: 450000000`, and its comment explains that as "~4 epochs of the 111.7M-token corpus". Both are v1-era facts. The README states that `runs/base_v2`, the project's headline model, trained on **2,499,805,184** tokens. That is $4,768 \times 524,288$, confirmed exactly against the committed `runs/base_v2/metrics.jsonl`, whose last logged step is 4,760.

So the v2 run was launched with `--max-tokens` overriding the config, and the anneal phase additionally needed `--data-dir data_anneal` on a `--resume`. Both flags exist in `train.py`. But **the exact commands are written down nowhere**: `docs/GPU_RUN.md` still documents only `python train.py --config configs/base.yaml`, which would train a different model on a different budget.

Someone cloning this repository and following the documentation cannot reproduce the model the repository is about. The fix is small: either commit a `base_v2.yaml` and an `anneal.yaml`

with the real values, or write the two literal commands into `GPU_RUN.md`. The training code is not at fault; the record of how it was invoked is missing.

Two stale comments in the configs

`configs/tiny.yaml`'s header says "~10M params"; the real figure is 13.01M, measured above. `configs/base.yaml`'s `grad_accum` comment says "effective ~524k tokens/step on a single 24GB GPU"; both real runs used an A100 SXM **80GB**. Neither misleads about anything load-bearing, and the README is scrupulous about the 13M figure elsewhere, so these are simply comments that were not revisited. `configs/medium.yaml`, by contrast, is exemplary: it explicitly flags its own `batch_size` as carried over from `base.yaml` and unverified against the larger model's memory footprint.

7.7 Distributed training

`is_dist()` checks the `WORLD_SIZE` environment variable that `torchrun` sets, so a plain `python train.py` takes the single-process path with zero distributed overhead and the same file works under `torchrun` unchanged. The backend is chosen by hardware: `nccl` on CUDA, `gloo` otherwise. Seeds are offset per rank (`seed = cfg.get("seed", 1337) + rank`) so ranks do not draw identical dropout masks.

Jargon: DDP (DistributedDataParallel)

Each GPU holds a full copy of the model and processes a different slice of the batch. After the backward pass the gradients are averaged across all GPUs, so every copy takes an identical optimizer step and stays in sync. It scales throughput nearly linearly.

The DDP path was never exercised

Both real runs used a single A100, and `docs/GPU_RUN.md`'s log shows `python3 train.py`, not `torchrun`. The DDP code is written carefully (the gradient-sync suppression above is a detail that only someone who has thought about DDP includes) but it is untested: no test covers it, and `world_size > 1` has never run. The `require_backward_grad_sync` attribute in particular is set on `model`, which is the DDP wrapper in that branch, which is correct, but "correct by reading" is not the same as "known to work". If asked, the honest answer is that the multi-GPU path is written but unproven.

8 Part VII: Evaluation

Four scripts, measuring four different things. Together they answer: is it a good language model (perplexity), is it writing Urdu at all (script mix), does it know anything (cloze), and was the tokenizer worth building (compare, already covered in Section 5.3).

8.1 eval/perplexity.py: the primary metric

Loads a checkpoint, memory-maps a .bin split, cuts it into blocks at fixed offsets, and averages the loss.

```
1 _, loss = model(x, y)
2 total_loss += loss.item() * x.numel()      # loss is a mean; re-weight by tokens
3 total_tokens += x.numel()
4 avg = total_loss / total_tokens
5 print(f"tokens={total_tokens} loss={avg:.4f} perplexity={math.exp(avg):.2f}")
```

Listing 20: The token-weighted average

Why the multiplication by x.numel() is correct and necessary

`F.cross_entropy` returns the *mean* loss over the batch's tokens. Averaging those means directly would be wrong if batches held different token counts (the last batch usually does). Multiplying each mean back out by its token count and dividing by the grand total gives a properly token-weighted average. It is a small thing that is frequently got wrong.

Unlike the loop's in-training eval, this script is fully deterministic: fixed starts, no shuffle, no sampling. It must reproduce exactly, which is what made the check below possible.

Verified, and it does not reproduce: 214.11, not the README's 248.8

The README states the tiny proof run's "final held-out perplexity (fuller eval on data/val.bin): **248.8**". Running the committed script against the committed checkpoint and the committed data on 2026-07-16 gives:

```
1 $ python -m eval.perplexity --ckpt runs/tiny/model.pt --bin data/val.bin
2 tokens=409600 loss=5.3665 perplexity=214.11
3
4 $ python -m eval.perplexity --ckpt runs/tiny/model.pt --bin data/val.bin \
5     --max-batches 100000          # the entire val split
6 tokens=1118208 loss=5.1762 perplexity=177.01
```

The script is deterministic, so this is not sampling noise. It was run twice, at the default coverage and over the whole validation set, to rule out the possibility that 248.8 corresponded to some other batch budget. It does not: more coverage moves the number *further* from the claim, not closer. The 248.8 was produced from different inputs.

The likely explanation is visible in the file timestamps. `runs/tiny/model.pt` was written at 20:21, but `data/val.bin` and `data/corpus_stats.json` were both rewritten at **21:46**, over an hour *later*. The validation set was regenerated after the tiny model was trained and evaluated, so the committed `val.bin` is not the one the 248.8 was measured on. That is also consistent with the direction of the gap: the current number is *better* than the claim, so nothing was inflated.

This is worth fixing rather than shrugging at, for two reasons. It is the one number in this repository found not to reproduce, in a project whose entire pitch is honest reporting; and a reader who checks it will find a mismatch and start wondering about the rest, which would be unfair to a project where everything else checked out to the digit. Re-run the eval and update the README, or state which `val.bin` the figure came from.

"Fuller eval" is actually the default, and it covers about 37% of val

The README calls the 248.8 a "fuller eval", implying more coverage than the in-training one. But the command it gives passes no flags, so it uses the defaults `--max-batches 200 --batch-size 8`. With the tiny model's 256-token blocks that is $200 \times 8 \times 256 = 409,600$ tokens, confirmed by the script's own output above. The committed `data/val.bin` is 2,236,612 bytes of uint16, so 1,118,306 tokens. The "fuller" eval therefore covers about **37%** of the validation set, and is capped by `--max-batches`, not by the data. It is fuller than the in-training eval (which uses `eval_iters` batches) but it is not a full pass, and the phrase invites the reader to think it is.

8.2 eval/script_mix.py: is it even writing Urdu?

A diagnostic no English-language project needs, and a genuinely clever cheap check.

```
1 for c in text:
2     if not c.isalpha(): continue           # skip digits and punctuation
3     if is_arabic_char(c): arabic += 1
4     elif c.isascii(): latin += 1
5     else: other += 1
```

Listing 21: Classifying generated characters

It generates from five fixed one-word Urdu prompts, cycled by index, at a temperature of 0.9 with `top_k` of 40, and reports the mean share of each script. It reuses `is_arabic_char` from the pipeline's own `normalize.py`, so the diagnostic and the corpus filter share exactly one definition of "Arabic script". That reuse is the right instinct.

What this metric actually catches

A model can have a plausible-looking perplexity while quietly degenerating into Latin text it absorbed from crawl contamination, or into punctuation loops. Perplexity would not obviously flag it; this would, instantly and in one number. The v2 model scores 99.75% Arabic script over 50 generations, which says the corpus filtering worked and the model learned to stay in Urdu.

The prompt is included in the text being scored

`text = tok.decode(out[0].tolist())` decodes the *whole* sequence, prompt included, and `script_shares` then counts all of it. Since every prompt is Urdu, the prompt's characters are counted as Arabic-script wins the model did not earn. With `max_new_tokens=80` against a one-word prompt the contamination is a few percent at most, so it cannot manufacture the 99.75% result. But it does mean the metric is biased upward by construction, and slicing off the prompt before scoring would be a two-line fix. Worth knowing before someone asks whether the 99.75% includes the prompt, because it does.

8.3 eval/cloze.py: does it know anything?

Jargon: Cloze test

A fill-in-the-blank. The model is given a stem (a sentence up to a blank) and several candidate completions, exactly one correct. It does not generate; it *scores* each candidate and the best-scoring one is its answer. This measures knowledge without requiring the model to be able to follow an instruction, which a 124M base model cannot.

Jargon: Few-shot prompting

Prepending a few solved examples to the prompt so the model can infer the pattern from context. `--shots 2` builds a prefix from the first two items (stem plus correct answer) and prepends it to every subsequent question. Those two items are then excluded from scoring, which is why 95 items yield 93 scored.

```

1 ctx_ids = tok.encode(context).ids
2 opt_ids = tok.encode(option).ids
3 ids = (ctx_ids + opt_ids)[-max_len:]
4 x = torch.tensor([ids[:-1]]); y = torch.tensor([ids[1:]])
5 mask_start = len(ids) - 1 - len(opt_ids)
6 y = y.clone()
7 y[0, :max(mask_start, 0)] = -100          # score ONLY the option's tokens
8 _, loss = model(x, y)
9 return loss.item()

```

Listing 22: Scoring one option

The masking is the heart of it, and it is right: everything before the option is set to -100 so the context contributes nothing to the loss, and only the candidate's own tokens are scored. `[-max_len:]` crops from the left if the few-shot prefix has overflowed the context window, keeping the option intact at the end where it must be.

The docstring says "total log-probability"; the code computes a length-normalized mean

The module docstring says the harness scores "the total log-probability of the completion tokens". It does not. `F.cross_entropy`'s default reduction is "mean", and with `ignore_index=-100` it averages over the non-ignored positions only. So `option_loss` returns the **mean per-token** loss of the option, not the sum.

The difference is not cosmetic. Total log-probability systematically favours *short* options, because every extra token adds more negative log-probability; mean per-token loss removes that length bias. Since the eval set's distractors are described as "grammatical and same-shape", the options are of similar length and the two would rank them nearly identically here, so the reported numbers are almost certainly unaffected.

Which makes this a documentation bug rather than a scoring bug, and arguably the code made the *better* choice than the docstring describes. But it is precisely the kind of detail an interviewer probes ("how do you handle length bias in multiple-choice scoring?"), and the answer should be "mean per-token, deliberately", not a docstring that says otherwise.

8.3.1 The eval set**Verified: 95 items, 3 options each, hand-written**

`evaldata/cloze.jsonl` was read directly: exactly 95 lines, every item with exactly 3 options, fields `id`, `stem`, `options`, `answer_idx`, `translation`. The README's "expanded from 45 to 95 items" is confirmed by the file.

evaldata/README.md still says 45 items

The eval set's own README describes "45 items" and "a probe for the phase-2 base model... the tiny CPU proof-run checkpoint scores near chance". The file has held 95 items since 2026-07-08, and the base model has been trained twice since. The top-level README is current; this one was not updated alongside it. It also still refers to the tiny run as the reference point, which is two runs out of date.

Every one of the 95 items has `answer_idx 0`

Verified by counting: the correct answer is the first option in all 95 items, with no exceptions. `evaldata/README.md` discloses this ("placed first in every item for authoring convenience") and correctly notes that the harness scores log-probabilities and never looks at position, so there is no leak through the scoring path.

That defence is sound, and this is not a live bug. But it costs something real: it makes a whole class of bug undetectable. If a refactor ever introduced a position bias, or if `pred` were computed wrongly and happened to return 0, the harness would score 100% and look like a triumph. A shuffled `answer_idx` would make that failure mode impossible and costs one script to produce. For a set explicitly built to be trusted for future comparisons, that is worth doing.

The Wikidata pipeline would contaminate this eval set, and the code says otherwise

This one matters, because it concerns a run that has not happened yet and can therefore still be fixed.

`pipeline/fetch_wikidata.py`'s docstring states the fact triples are "using different entities than the eval set's own 45 items", and `docs/DATA_LICENSES.md` repeats it: "distinct entities from the eval set's own 45 cloze items". **Nothing in the code enforces this, and it is demonstrably false.**

The `capital` query is `?subject wdt:P36 ?object . ?subject wdt:P31 wd:Q6256`, which selects the capital of *every country*. Pakistan is a country. So the query returns Pakistan/Islamabad, and `read_wikidata_facts` renders it into a declarative Urdu sentence of the form "the capital of Pakistan is Islamabad".

Cloze item `id 1`, read directly from `evaldata/cloze.jsonl`, glosses in its own `translation` field as: "The capital of Pakistan is Islamabad." Item 21 is "Urdu is the national language of Pakistan", which the `official_language` (P37) query covers the same way.

So training on the Wikidata source as configured would put at least one eval item's exact fact, in near-identical declarative phrasing, directly into the training data. That is textbook train/test contamination, and it lands on the *one metric* the `medium` run exists to improve. A cloze gain measured that way would be partly unearned, and the project's whole argument (that fact recall needs more parameters, per Roberts et al.) is exactly the kind of claim that a contaminated eval would appear to confirm.

Two honest caveats, in the project's favour. First, the contamination is small: two of 95 items are affected by the four relations queried. Second, Urdu Wikipedia certainly already contains prose stating Pakistan's capital, so the eval set was never contamination-free in the strict sense, and no one claims a base model's cloze score is a clean held-out measurement. What is specifically wrong is the *documented claim of entity-distinctness*, which is asserted twice and enforced nowhere.

The fix is easy and should happen before the `medium` run: load `evaldata/cloze.jsonl`, extract its subjects, and filter them out of the fetched triples, with a test pinning it. Then the docstring becomes true.

8.4 `eval/plot_loss.py` and `sample.py`

`plot_loss.py` reads `metrics.jsonl` and renders the curve to a PNG with matplotlib, using the Agg backend so it works on a headless box. It keys on the presence of "loss" versus "val_loss" to separate the two series, which is exactly the shape the training loop writes.

`sample.py` loads a checkpoint and prints continuations for three default Urdu prompts (transliterated: *pakistan ek, urdu zaban, aaj mausam*).

sample.py duplicates load_model verbatim from eval/_shared.py

The two functions are identical, line for line: same `torch.load`, same `ModelConfig(**ck["config"])`, same `.eval()`, same `return`. `eval/cloze.py` and `eval/script_mix.py` both import the shared one; `sample.py` defines its own copy instead. Presumably `sample.py` sits at the repo root rather than inside `eval/` and the import was avoided. `conftest.py` already puts the repo root on `sys.path`, and the eval scripts import `pipeline.common` across packages anyway, so `from eval._shared import load_model` would work. Fifteen duplicated lines is trivial, but it is the one place where the repository's otherwise good factoring slips.

9 Part VIII: What actually happened when it ran

Three real runs. Everything in this part was read out of the committed `metrics.jsonl` files, not out of the README.

9.1 The tiny CPU proof run

The point was never capability. It was to prove the pipeline end to end on an 8-core CPU: real corpus, real tokenizer, real training loop, a loss that actually falls, and a checkpoint that loads.

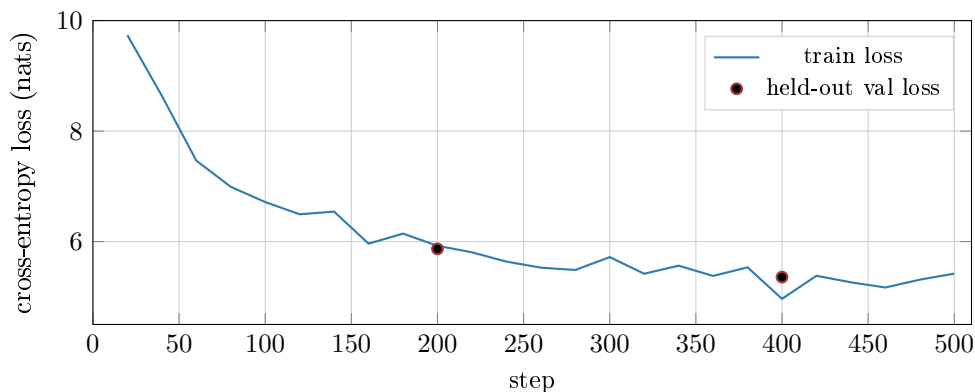


Figure 11: The tiny run, drawn from the committed `runs/tiny/metrics.jsonl`. 500 steps, 4.10M tokens. The curve is still clearly falling at the end; the run was stopped at a checkpoint boundary, not at convergence.

Verified against the log: first logged train loss **9.7365** at step 20, last **5.4204** at step 500; val loss **5.8678** at step 200 and **5.3572** at step 400. These match the README's "9.74 to 5.42" and its val perplexities (352.5 and 212.1, which are $e^{5.8678}$ and $e^{5.3572}$) exactly.

Note the honesty the curve itself demands: the last three train points are 5.169, 5.311, 5.420, so the final logged value is a *local uptick*, not the minimum. The README quotes 5.42, the final value, rather than cherry-picking the 4.964 at step 400. That is the correct choice and the harder one.

9.2 Base v1: the first real GPU run

Date and hardware	2026-07-07, rented A100 SXM 80GB (RunPod)
Steps and tokens	858 steps, 449.8M tokens, roughly 4 epochs of the v1 corpus
Measured throughput	~99,470 tokens/second sustained
Wall clock and cost	about 75 minutes, about \$1.90
Loss	first logged 9.5863 (step 20), last logged 3.5087 (step 840)
Held-out perplexity	38.02

Table 6: Base v1. Throughput and losses read from `runs/base/metrics.jsonl`.

The README's "loss fell 7.35 to 3.51" understates the starting loss

7.3513 is a real number in `runs/base/metrics.jsonl`, but it is the value at **step 100**, not the start. The first logged loss is **9.5863** at step 20. The run's honest range is 9.59 to 3.51.

Two things make this worth flagging despite being minor. It is inconsistent with the project's own reporting of v2 in the same README paragraph, which correctly quotes 9.63 (the step-20 value) as that run's starting loss, so the two runs are described on different bases and are not directly comparable as written. And it understates the model's actual improvement,

which is the opposite of the direction people usually err, so this reads as an editing slip rather than anything else. The fix is one number.

v1 has no in-training validation numbers at all

`runs/base/metrics.jsonl` contains **zero** `val_loss` records. The reason is arithmetic: `eval_interval` is 1000 in `configs/base.yaml`, and the run's total was 858 steps, so the evaluation branch never fired once. The 38.02 perplexity in the README came from a separate `eval.perplexity` invocation afterwards, which is fine and is what that script is for. But it means the v1 run flew blind: there was no signal during those 75 minutes that would have revealed overfitting or divergence. v2, at 4,768 steps, got four val checks. Setting `eval_interval` above the run's own step count is a small planning slip worth noticing.

9.3 Base v2: the headline model

The v2 run is the project's real result: roughly 7.7x more unique tokens than v1, a token/parameter ratio of about 20.2 against v1's 0.9, and a two-phase schedule ending in a Wikipedia-only anneal.

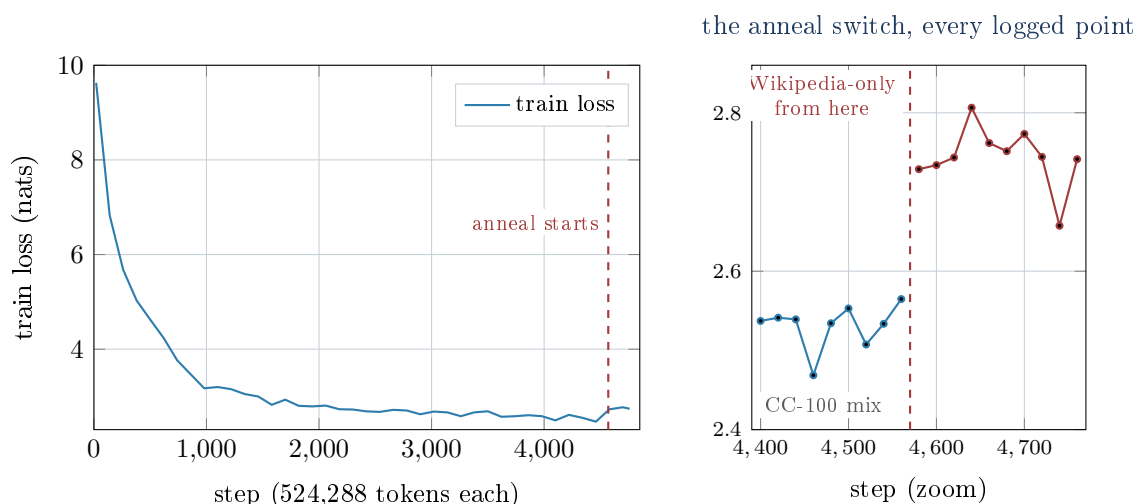


Figure 12: The v2 run, every 6th logged point from the committed `runs/base_v2/metrics.jsonl`, plus the final point. The loss step at the anneal boundary is real data, not an annotation.

Verified: the anneal is visible in the log, exactly where it should be

The README says the loss "rose briefly and expectedly at the phase-1 to anneal data-source switch, then settled". The committed log confirms this precisely. Steps 4400 to 4560 sit at 2.47 to 2.56; step 4580 jumps to 2.73 and the run finishes in the 2.74 to 2.81 band. The jump lands between steps 4560 and 4580.

That is exactly where arithmetic predicts it. The main phase was about 2.4B tokens, which at 524,288 tokens per step is roughly step 4,578. So the log independently corroborates the README's account of a two-phase run, and the rise is what a data-distribution change looks like, not instability: Wikipedia-only text is a different (and for this model, harder) distribution than the CC-100 mix it had adapted to.

step	tokens	val loss	val perplexity
1000	524,288,000	3.9621	52.57
2000	1,048,576,000	3.5324	34.21
3000	1,572,864,000	3.3818	29.42
4000	2,097,152,000	3.3074	27.31

Table 7: Every in-training validation record from `runs/base_v2/metrics.jsonl`, matching the README’s 52.57 and 27.31 endpoints. The curve is flattening: doubling the tokens from 1B to 2B bought 6.9 perplexity points, while the next 1B bought 2.1.

9.4 v1 against v2, and the honest reading

Metric	v1 (2026-07-07)	v2 (2026-07-08)
Unique training tokens	111.7M	862.4M
Tokens seen	449.8M	2,499,805,184
Tokens per parameter	0.9	20.2
Held-out perplexity	38.02	28.93
Script mix (Arabic)	99.43% (20 gens)	99.75% (50 gens)
Cloze, 45-item set	34.9% (15/43)	39.5% (17/43)
Cloze, 95-item set	not measured	45.2% (42/93)
GPU time and cost	~75 min, ~\$1.90	6h 39m, ~\$13.06

Table 8: The two GPU runs. Cloze figures are README-reported; the perplexity, script-mix and cloze numbers for the base models could not be re-run here (see below).

The project’s own reading of these numbers is the correct one

The README does something unusual and right: it refuses to claim the cloze improvement. 34.9% to 39.5% is a 2-item difference on a 43-item set whose standard error is roughly ± 7 points, so it sits well inside noise. The README says so in as many words, rather than presenting “+4.6 points” as a result.

It then explains *why* cloze barely moved while perplexity fell 24%, and the explanation is both correct and well-sourced: a 124M model has a real ceiling on how many discrete facts it can store, and that capacity scales with parameter count, not with how much text it reads (Roberts et al., 2020). So 7.7x more text made it more *fluent* (which is what perplexity measures) without making it know more (which is what cloze measures). The corpus expansion did not fail; it did exactly what more text does.

That diagnosis then drives the plan: the `medium` preset exists because more parameters is the lever that moves fact storage, and the eval set was widened from 45 to 95 items to cut the noise band from ± 7 to ± 5.2 precisely so the next comparison can be trusted. Identifying that your headline metric is too noisy to support your next claim, and fixing the metric before running the experiment, is the most mature single decision in this repository.

What could not be verified here, and why

The base models’ own evaluation numbers (perplexity 38.02 and 28.93, script mix, cloze) were **not** re-run for this document, and are reported on the README’s authority. Two hard blockers:

- `data/train.bin` is gitignored (214MB for v1, 1.7GB for v2, both over GitHub’s 100MB per-file limit), and the v2 corpus cannot be rebuilt without re-downloading roughly 1.3GB of raw sources and running a multi-hour pipeline.

- The **base** weights are present via Git LFS, but evaluating a 124M model over the validation set on this 8-core CPU is hours of work, and the perplexity figures were measured against the *v2* validation set, which is not the committed `data/val.bin` (that one is *v1*'s, as the `corpus_stats.json` shows).

What *was* verified from the committed artifacts: the training losses, throughputs, step and token counts, the anneal boundary, all four presets' parameter counts, the full test suite, the eval set's shape, and the tokenizer's headline number. The base models' downstream eval metrics remain README-attributed, and are labelled as such in Table 8.

`docs/GPU_RUN.md` was never updated after either run and is now actively misleading

The document opens with "phase 2 is a budget decision, not an engineering one", estimates throughput at "roughly 25k-40k tokens/s... taking a conservative 30k tok/s", and states flatly: *"These are estimates, not measured; the only measured throughput in this repo is the CPU proof run."*

Two GPU runs have since happened, and the committed logs show a real sustained **99,470** tok/s (*v1*) and **105,000** tok/s (*v2*). The "conservative" estimate was low by a factor of about 3.3. Every downstream figure in that document inherits the error: it projects the 450M-token run at ~4.2 hours for about \$6, where *v1* actually did 449.8M tokens in about 75 minutes for about \$1.90.

Its token-budget section is *v1*-era throughout ("our cleaned corpus is 111.7M training tokens... only about 0.9 tokens per parameter"), which the *v2* corpus superseded by 7.7x, and its "Add more data before spending GPU money" advice was taken and completed on 2026-07-08. It even dates its own run log 2026-07-06 where the README says the *v1* run was 2026-07-07. The parts of that document written *after* the run are excellent and are the reason this is worth fixing rather than deleting: the RunPod SSH key injection trap, the warning that installing the pinned `torch==2.12.1` would silently replace the pod's CUDA build with a CPU-only one, the `tmux` detach pattern, and the note that SSH cannot stop the billing meter so you must Terminate (not Stop) from the dashboard. That is hard-won operational knowledge that no tutorial contains.

The document is simply two documents stapled together: a pre-run plan whose predictions were falsified, and a post-run operations log that is entirely accurate. Splitting them, and replacing the estimates with the measured numbers, would turn the project's weakest document into one of its strongest. As it stands, a reader who trusts it will believe the model is data-limited at 111.7M tokens and that an A100 does 30k tok/s, and both were disproven by this project's own runs.

10 Part IX: The planned `medium` run

Three pieces are built and tested but have never been run: the `medium` preset (336.38M parameters, verified above), the Wikidata fact pipeline, and the widened 95-item eval set. The reasoning chain behind them is the strongest argument in the project, and it is worth being able to state it in one breath:

The argument, end to end

v2's corpus expansion cut perplexity 24% but left cloze inside the noise band. Roberts et al. (2020) found fact-storage capacity scales with *parameter count*, not with training-text volume. Therefore more text cannot fix fact recall, and the next lever must be a bigger model plus training data shaped as dense declarative facts. Hence `medium` (2.7x the parameters) and the Wikidata triple pipeline. And because the old 45-item eval could not resolve the difference anyway, widen it to 95 first.

The cost estimate in the README is explicitly labelled a projection, and its uncertainties are named rather than buried: throughput is scaled from v2's measured $\sim 105,000$ tok/s by the parameter ratio ($336/124 \approx 2.71$, projecting $\sim 38,700$ tok/s), which assumes compute-bound scaling; and `configs/medium.yaml`'s `batch_size: 32` is inherited from `base.yaml` and unverified against a model whose activations are roughly 2.3x larger, where `base` already used 36GB of 80GB. The recommendation to smoke-test a few hundred steps before committing the $\sim \$40$ -45 budget is the right one.

The one thing to fix before launching

The Wikidata contamination described in Section 8.3. The `medium` run exists to move the cloze number; feeding it training data containing at least two of the eval set's own facts, while the code's own docstring claims the entities are distinct, would compromise the exact measurement the run is for. It is perhaps thirty lines of filtering and a test.

11 Part X: Honest findings, collected

Every finding in this document, gathered in one place and ordered by how much it would cost to be surprised by it. Nothing here is fatal; this is a well-built repository and the headline claims that could be checked all checked out. But an interviewer will find some of these, and it is better to get there first.

11.1 Substantive: worth fixing before the next run

1. **Upsampling repeats documents back-to-back inside a single context window** (Section 5). `buf.extend(ids * reps)` lays three copies of a Wikipedia paragraph down contiguously, and at ~ 70 tokens per paragraph all three fit inside one 1024-token block. This teaches copying rather than reweighting the distribution, and it shipped in the run that produced the headline model. Cheap to fix at the document-list level.
2. **The Wikidata fact pipeline would contaminate the cloze eval set, and two documents claim otherwise** (Section 8.3). The P36 query returns every country's capital, including Pakistan's, which is cloze item 1 verbatim. Nothing enforces the documented entity-distinctness. Must be fixed before the medium run, whose entire purpose is to move that metric.
3. **`configs/base.yaml` cannot reproduce `runs/base_v2`, and nothing records what would** (Section 7). The config says 450M tokens; the run did 2.5B plus an anneal on a different data directory. The flags exist; the invocations are written down nowhere.
4. **The tiny run's reported perplexity does not reproduce** (Section 8). README says 248.8; the committed script on the committed checkpoint and data gives **214.11** at the default coverage and **177.01** over the whole val split, so no batch budget produces the claim. The timestamps suggest `val.bin` was regenerated an hour after the figure was measured. The only number in the project found not to reproduce.
5. **`docs/GPU_RUN.md` is a pre-run plan whose predictions were falsified, never updated** (Section 9). It estimates 30k tok/s where the real runs measured 99.5k and 105k, states no GPU throughput has been measured when two runs have, and reasons throughout from the superseded 111.7M-token v1 corpus. Its post-run operations notes, by contrast, are excellent.

11.2 Real but contained

6. **The LSH index and the exact-hash set are held entirely in RAM** (Section 4.3), the one place the streaming discipline breaks. The README names this itself.
7. **`generate` has no KV cache**, so sampling is quadratic (Section 6). Fine for this project's usage, disqualifying for serving, and the reason the GQA branch has nothing to optimize.
8. **GQA is implemented with `repeat_interleave`**, which materializes the expanded tensors and therefore saves no memory. It is a correctness shim, not an optimization, and no preset enables it. The README's word "capable" is exactly right.
9. **The DDP path has never been executed** (Section 7). Written carefully, including the gradient-sync suppression, but unproven and untested.
10. **The epoch wrap does not reshuffle and block offsets are fixed**, so v2's roughly 3 passes saw an identical block sequence each time.
11. **Stage resume is coarse**: markers are written only on stage completion, so a crash 90% through extraction redoes all of it.
12. **v1 never ran a single in-training validation**, because `eval_interval` (1000) exceeded the run's total steps (858).

13. **script_mix scores the prompt along with the generation**, biasing the Arabic share upward. Too small to explain the 99.75%, but real.
14. **The tokenizer comparison samples val-then-train**, and is held-out only because val happens to be big enough, not because anything enforces it.

11.3 Documentation and hygiene

15. **The README's tokenizer command omits --max-chars 800000** and as written yields 5.14x, not the 5.23x printed beside it (Section 5.3). The number is genuine and reproduces exactly with the flag.
16. **cloze.py's docstring says "total log-probability"** but the code computes a length-normalized mean, which is the better choice, undocumented (Section 8.3).
17. **train.py's docstring promises RNG restoration** that the checkpoint does not contain. Behaviourally near-harmless; the claim is the defect.
18. **README's v1 "loss fell 7.35"** is the step-100 value, not the step-20 start of 9.5863, and is inconsistent with how v2 is reported in the same paragraph.
19. **evaldata/README.md still says 45 items**; the file has held 95 since 2026-07-08. **DATA_LICENSES.md** likewise says "the eval set's own 45 cloze items".
20. **"Fuller eval" covers about 37% of val**, capped by --max-batches, not by the data.
21. **All 95 cloze items have answer_idx 0**. Disclosed, and the harness is position-blind, but it makes a whole class of bug undetectable.
22. **num_params(non_embedding=True) has two identical branches** (Section 6); collapses to one line.
23. **sample.py duplicates load_model verbatim** from eval/_shared.py.
24. **build_minhash's num_perm parameter is a trap**: the module's _A/_B are fixed at 64, so any other value would silently mismatch.
25. **<|pad|> and <|unk|> are structurally unreachable** (Section 5). Two inert vocabulary slots.
26. **The RoPE cache keys on T but ignores it**, so **generate** builds one identical full-size cache entry per token sampled.
27. **Stale config comments**: **tiny.yaml** says "~10M params" (real: 13.01M); **base.yaml** says "single 24GB GPU" (real: A100 80GB).
28. **data/corpus_stats.json describes v1 only**; there is no committed stats file for the v2 corpus the headline model trained on.
29. **Pinned versions do not match the installed environment** (Section 12). All 13 tests pass anyway.

11.4 What is genuinely strong

It would be a distorted document that listed thirty findings and no strengths, so:

- **The headline claim is real and reproduces to the digit.** 806,495 chars, 197,758 tokens, 4.078 chars/token, GPT-2's 1,034,831 and 0.779, gain 5.23x. Every figure, exactly.
- **The MinHash vectorization** is a real 10x on a real bottleneck, with a correct and articulable reason why swapping the hash function is safe.
- **The refusal to claim the cloze improvement**, followed by widening the eval set to fix the noise band *before* running the experiment that needs it.

-
- **The refusal to shrink the vocabulary** to make "10M params" true.
 - **The 8/3 SwiGLU ratio and the 14-layer consequence**, worked out rather than copied.
 - **Token-budgeted training**, which makes the Chinchilla reasoning directly expressible in the config and comparable across hardware.
 - **The causal-masking test**, which tests the property by experiment and includes the negative assertion that stops it being a tautology.
 - **The ZWNJ decision and the harakat decision**, both of which require actually knowing something about Urdu orthography rather than about machine learning.
 - **Validation is never upsampled or filtered**, and the tokenizer is trained on train only. Both leaks avoided quietly.
 - **The committed metrics.jsonl files**, which are what made most of this document checkable rather than merely repeatable.

12 Part XI: Reproduction, and what was verified

12.1 Dependency roles

Package	Pinned	Role
torch	2.12.1	the tensors, autograd, AdamW, SDPA/Flash, DDP
tokenizers	0.23.1	Hugging Face BPE trainer and encoder (Rust)
numpy	2.5.1	memmap, the vectorized MinHash, batch assembly
mwparserfromhell	0.7.2	MediaWiki wikitext parsing
datasketch	2.0.0	MinHash LSH index
xxhash	3.8.1	fast deterministic shingle hashing
PyYAML	6.0.3	config loading
matplotlib	3.11.0	the loss-curve PNG
pytest	9.1.1	the 13 tests
tqdm	4.68.3	declared but never imported anywhere

Table 9: `requirements.txt`. Note the last row.

The dependency list is notably restrained

There is no `transformers`, no `datasets`, no `accelerate`, no `lightning`, no `wandb`. The modeling is hand-written, the data pipeline is hand-written, the training loop is hand-written, and metrics go to a JSONL file. The only heavyweight import is PyTorch itself. For a project whose purpose is to demonstrate that the author can build this stack rather than assemble it, that restraint *is* the deliverable.

`tqdm` is declared and never used

`grep` finds no import of it anywhere in the repository. It is one line in `requirements.txt` and harmless, but a reviewer reading the dependency list will form an expectation of progress bars that the code does not meet.

The pinned versions are not the versions this was verified against

`requirements.txt` pins `torch==2.12.1`, `tokenizers==0.23.1`, `numpy==2.5.1`. The environment on this machine has torch 2.10.0+cu128, tokenizers 0.21.1, numpy 1.26.4, and **all 13 tests pass and every measurement in this document reproduced** regardless. So the code is more version-tolerant than the pins suggest. `docs/GPU_RUN.md` already documents the one place this bites: on a RunPod pod, installing the pinned torch would silently replace the working CUDA build with a CPU-only wheel, and the workaround is to filter torch out of the requirements before installing. Exact pins plus a documented "except do not install this one" is a slightly awkward combination; the pins are doing less work than they appear to.

12.2 What was verified for this document, and how

Claim	Status	Evidence
Tokenizer: 5.23x vs GPT-2, 4.078 vs 0.779 chars/token, on an 806,495-char sample	verified	ran <code>eval.tokenizer_compare</code> against a cached real GPT-2 tokenizer, <code>--max-chars 800000</code> ; every digit matched
13 tests pass (5 model, 8 pipeline)	verified	<code>pytest tests/ -q</code> : 13 passed in 55.42s
Preset parameter counts 13.0 / 35.7 / 123.7 / 336.38M	verified	instantiated all four at vocab 32000 and counted
v1 corpus funnel (3.7M paragraphs, 31.94% dedup, 2,489,976 / 25,106 split)	verified	read <code>data/corpus_stats.json</code>
Tiny run: loss 9.74 to 5.42, val 5.87 to 5.36	verified	read <code>runs/tiny/metrics.jsonl</code>
v1: 858 steps, 449.8M tokens, ~99,470 tok/s	verified	read <code>runs/base/metrics.jsonl</code>
v2: 4,768 steps, 2,499,805,184 tokens, ~105,000 tok/s, loss to ~2.74	verified	read <code>runs/base_v2/metrics.jsonl</code> ; token count is exactly 4768×524288
v2 anneal caused a brief loss rise	verified	visible at step 4560 to 4580 in the log, where arithmetic predicts it
v2 val perplexity 52.57 to 27.31	verified	the four <code>val_loss</code> records in the v2 log
Cloze set is 95 items, 3 options	verified	read <code>evaldata/cloze.jsonl</code>
Tiny final perplexity 248.8	contradicted	re-ran the script two ways: 214.11 at the default coverage, 177.01 over the whole val split. Neither is 248.8
v1/v2 base perplexity (38.02 / 28.93), script mix, cloze accuracy	README-attributed	<code>train.bin</code> gitignored, v2 val set not committed, CPU infeasible
v2 corpus table (11.0M CC-100 docs, 37.5% dedup, 13.3M train docs)	README-attributed	no committed stats file for v2
GPU costs (\$1.90, \$13.06) and wall-clock	README-attributed	billing is external to the repository
890 Wikidata triples fetched, 865 surviving dedup	README-attributed	requires a live SPARQL call

Table 10: The verification log for this document. "Verified" means the code was run or the committed artifact was read on 2026-07-16.

12.3 Running it yourself

```

1 # tests
2 python -m pytest tests/ -q
3
4 # the headline claim (note the flag the README omits)
5 python -m eval.tokenizer_compare \
6     --gpt2 ~/.cache/huggingface/hub/models--gpt2/snapshots/*/tokenizer.json \
7     --max-chars 800000
8
9 # the tiny model, on the committed checkpoint and val split
10 python -m eval.perplexity --ckpt runs/tiny/model.pt --bin data/val.bin
11 python sample.py --ckpt runs/tiny/model.pt --seed 7

```

Listing 23: The paths that were exercised while writing this document

Everything above works from a fresh clone with `pip install -r requirements.txt`, because `data/val.bin`, `tokenizer/tokenizer.json` and `runs/tiny/model.pt` are all committed. Rebuilding the corpus requires roughly 1.3GB of raw downloads and several hours; the `base` weights need `git lfs pull`.

The last word

The claim this project is built on, that an Urdu-native tokenizer is worth building because English tokenizers shred the script, was checked here against a real GPT-2 tokenizer and reproduced exactly: 5.23x. The engineering underneath it is sound, the scaling reasoning is correct and well-sourced, and the reporting is unusually honest, including where it is unflattering.

The findings in Part X are almost entirely about *documentation drifting behind code*: a GPU-run plan never updated after the run, configs that no longer describe the model that shipped, docstrings promising slightly more than they deliver, and a perplexity measured against a validation set that was later regenerated. The code is in better shape than the prose around it, which is the opposite of the usual failure and is a much easier problem to fix.